Министерство науки высшего образования Российской Федерации

федеральное государственное автономное образовательное учреждение высшего образования

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

(Университет ИТМО)

Факультет прикладной информатики

Образовательная программа Информационная безопасность

Направление подготовки (специальность) 09.03.03 - Прикладная информатика

О Т Ч Е Т

по домашней работе №1

Тема задания: CSS Flexbox, CSS Grid

Обучающийся Корольков Артем Алексеевич, К3440

Проверил Добряков Д.И., преподаватель ФПИН, Университет ИТМО

Дата 16.10.2026

Санкт-Петербург

2026

СОДЕРЖАНИЕ

1. Цель и содержание работы....3
2. CSS Flexbox: тренажёр Flexbox Froggy.... 4
2.1. Описание тренажёра.... 4
2.2. Изученные свойства CSS....4
2.3. Темы уровней и результат прохождения....4
3. CSS Grid: тренажёр Grid Garden....18
3.1. Описание тренажёра.... 18
3.2. Изученные свойства CSS Grid....18
3.3. Темы уровней и результат прохождения.... 19
4. Основы Git: тренажёр Learn Git Branching....35
4.1. Описание тренажёра.... 35
4.2. Изученные команды Git.... 35
Дополнительные команды среды тренажёра....36
4.3. Результат прохождения.... 36
5. Заключение.... 49

1. Цель и содержание работы

Цель работы — закрепить практические навыки построения веб-макетов с помощью CSS Flexbox и CSS Grid, а также освоить базовые действия системы контроля версий Git в интерактивной среде.

В ходе домашней работы требовалось пройти три тренажёра: Flexbox Froggy, CSS Grid Garden и Learn Git Branching. Первые два тренажёра помогают закрепить свойства CSS на наглядных задачах с перемещением объектов по игровому полю. Третий тренажёр моделирует историю коммитов и позволяет отработать ветвление, слияние и обмен изменениями с удалённым репозиторием.

В отчёт включены описание каждого тренажёра, пояснение изученных свойств и команд, краткий итог прохождения, а также все предоставленные скриншоты. Под каждым скриншотом размещена подпись с номером рисунка и обозначением соответствующего уровня или итогового экрана.

2. CSS Flexbox: тренажёр Flexbox Froggy

2.1. Описание тренажёра

Flexbox Froggy — интерактивный тренажёр по CSS Flexbox. Пользователь управляет положением лягушек на игровом поле, вводя CSS-свойства контейнера и отдельных элементов. Каждый уровень задаёт определённое расположение объектов, поэтому результат изменения свойства сразу виден на экране.

Flexbox предназначен для распределения элементов вдоль главной оси и выравнивания их по поперечной оси. Главная ось зависит от значения flex-direction. При переносе элементов на несколько строк становится важно также управлять распределением самих строк.

2.2. Изученные свойства CSS

Свойство	Назначение и пояснение
display: flex	Создаёт flex-контейнер; его непосредственные дочерние элементы становятся flex-элементами.
justify-content	Распределяет элементы вдоль главной оси. Значения: flex-start, flex-end, center, space-between, space-around, space-evenly.
align-items	Выравнивает все элементы по поперечной оси. Примеры: stretch, flex-start, flex-end, center, baseline.
flex-direction	Задаёт направление главной оси: row, row-reverse, column или column-reverse.
order	Меняет визуальный порядок одного элемента. Меньшее значение располагается раньше; по умолчанию используется 0.
align-self	Переопределяет выравнивание по поперечной оси для конкретного элемента, заданное через align-items.
flex-wrap	Определяет перенос элементов на новые строки или колонки: nowrap, wrap, wrap-reverse.
flex-flow	Краткая запись двух свойств: flex-direction и flex-wrap, например flex-flow: column wrap.
align-content	Распределяет свободное пространство между flex-строками/колонками, когда элементов несколько рядов; например flex-start, flex-end, center, space-between, space-around, stretch.

2.3. Темы уровней и результат прохождения

Последовательность уровней постепенно усложняет задачу: сначала используется justify-content для размещения элементов по главной оси, затем align-items и flex-direction, далее — индивидуальное управление порядком и

выравниванием, перенос элементов и распределение нескольких строк. В заключительной части требуется комбинировать несколько свойств одновременно.

Уровни	**Основной изучаемый приём**
1–4	Размещение элементов по главной оси с помощью justify-content.
5–7	Выравнивание по поперечной оси и сочетание justify-content с align-items.
8–13	Смена направления потока с flex-direction и комбинирование выравниваний.
14–16	Изменение порядка элемента через order и индивидуальное выравнивание через align-self.
17–20	Перенос элементов с flex-wrap и сокращённой записью flex-flow.
21–24	Управление распределением нескольких строк с align-content и комплексные комбинации свойств.

Результат: по предоставленным материалам зафиксировано прохождение 24 уровней Flexbox Froggy. Скриншоты приведены ниже в последовательности от первого до двадцать четвёртого уровня.

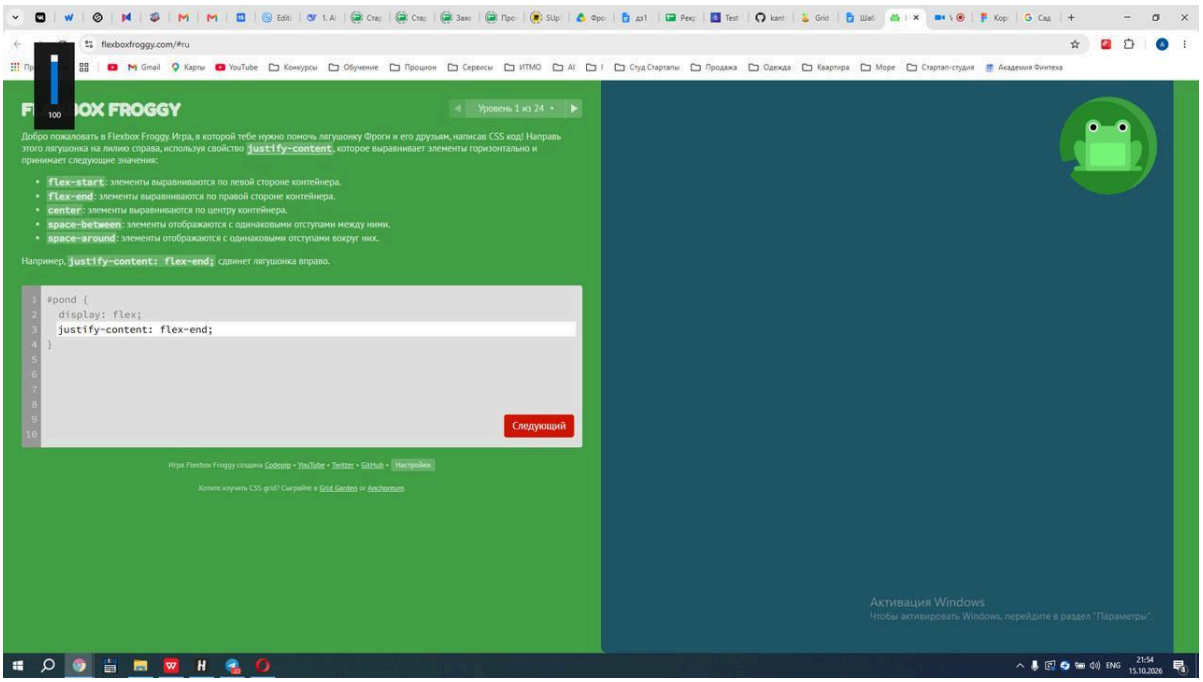


Рисунок 1 — Flexbox Froggy, уровень 1

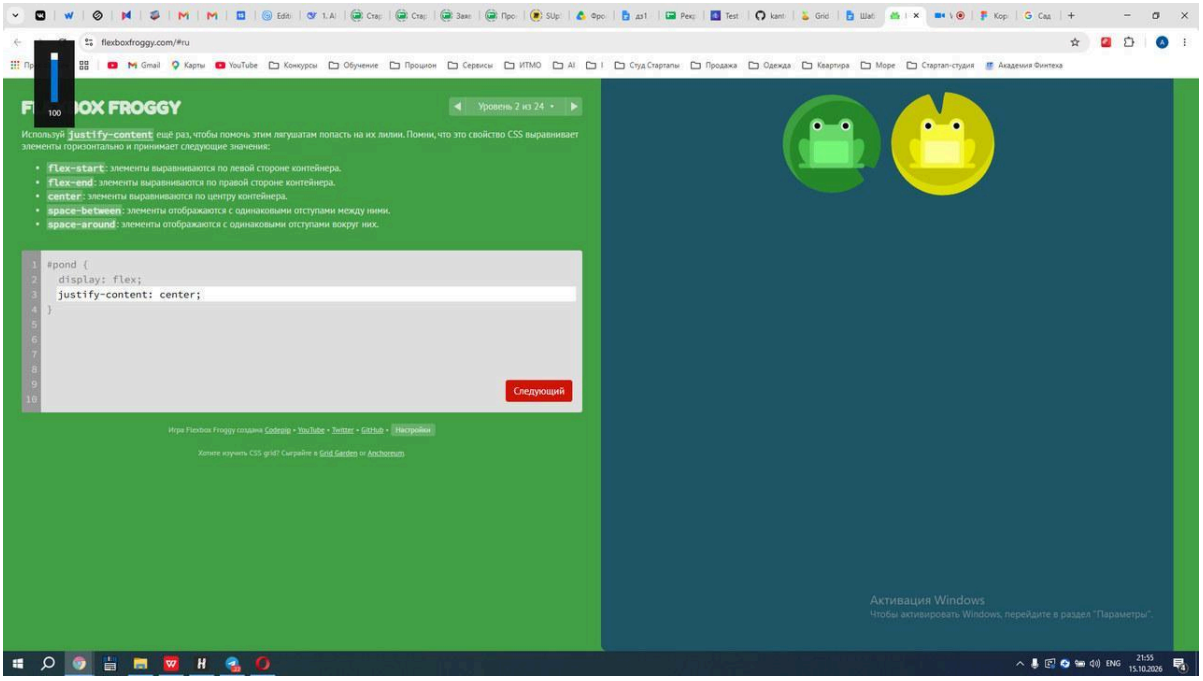


Рисунок 2 — Flexbox Froggy, уровень 2

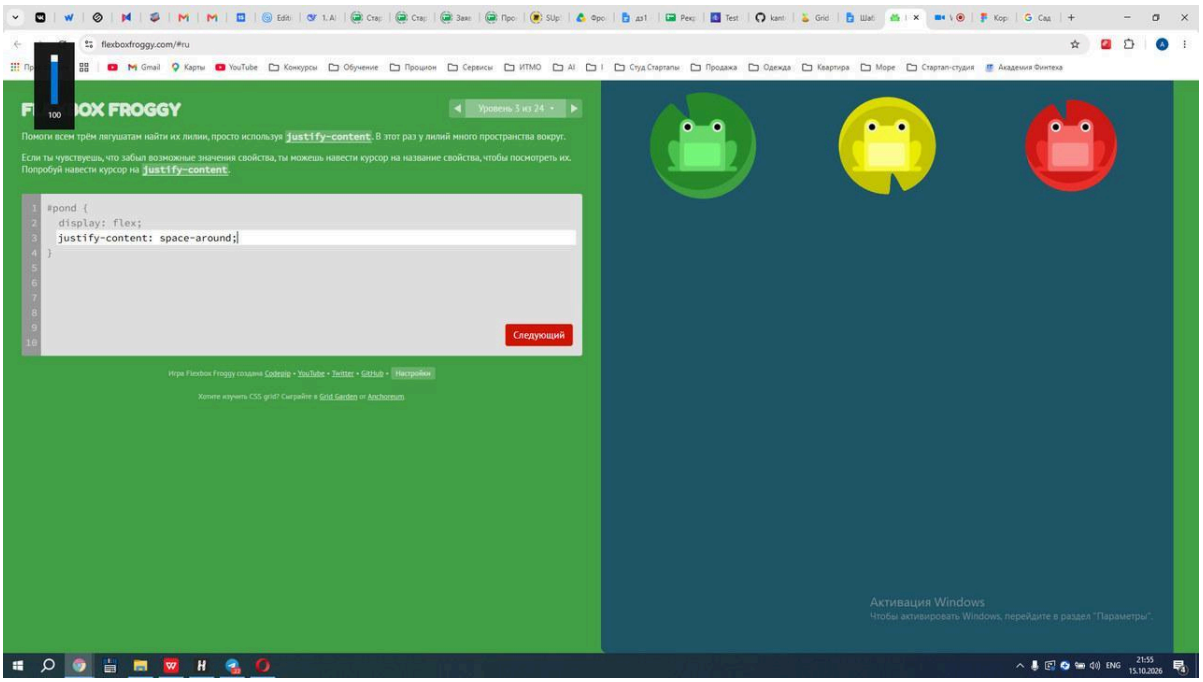


Рисунок 3 — Flexbox Froggy, уровень 3

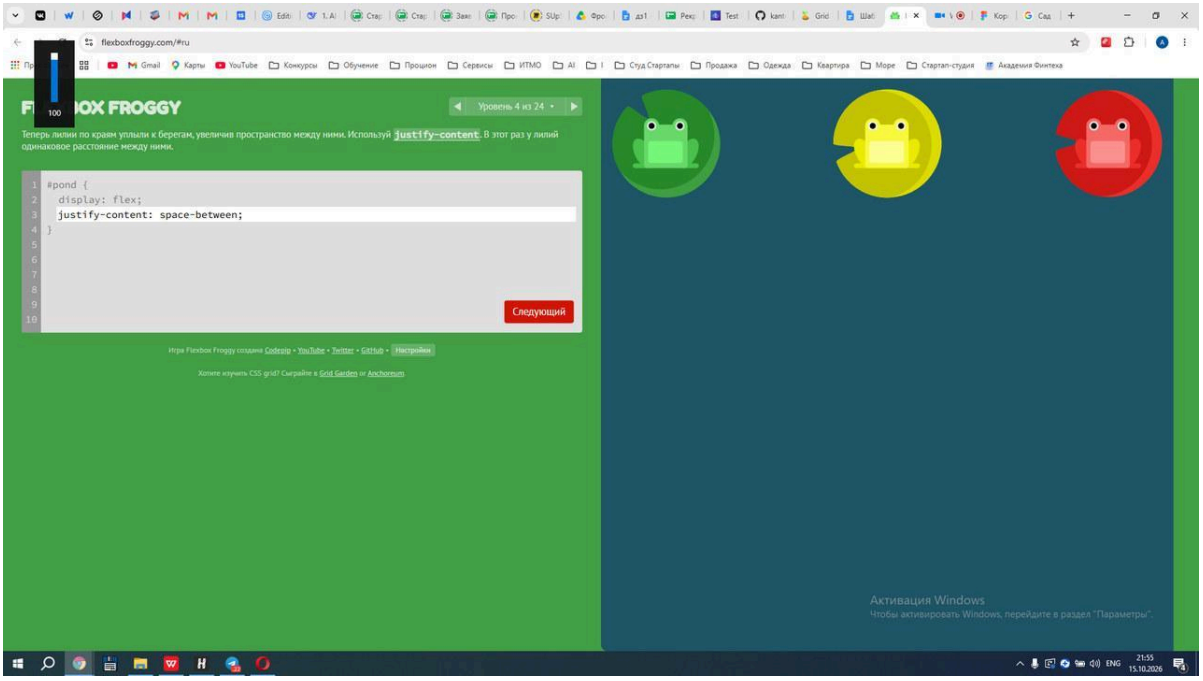


Рисунок 4 — Flexbox Froggy, уровень 4

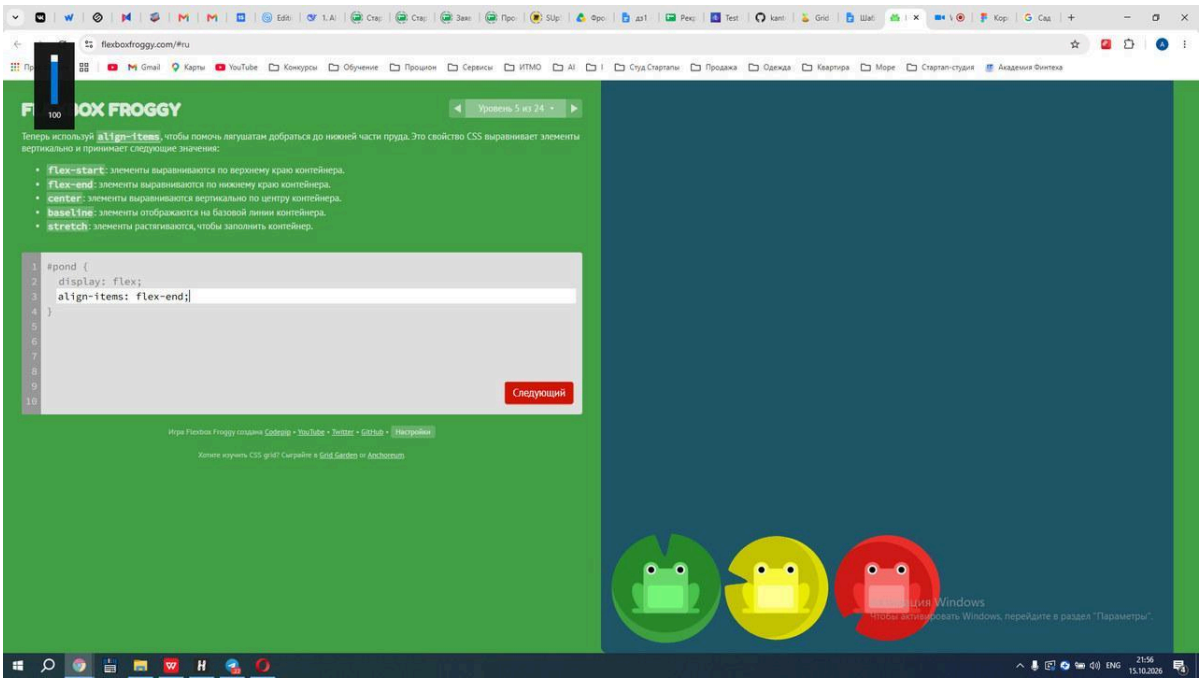


Рисунок 5 — Flexbox Froggy, уровень 5

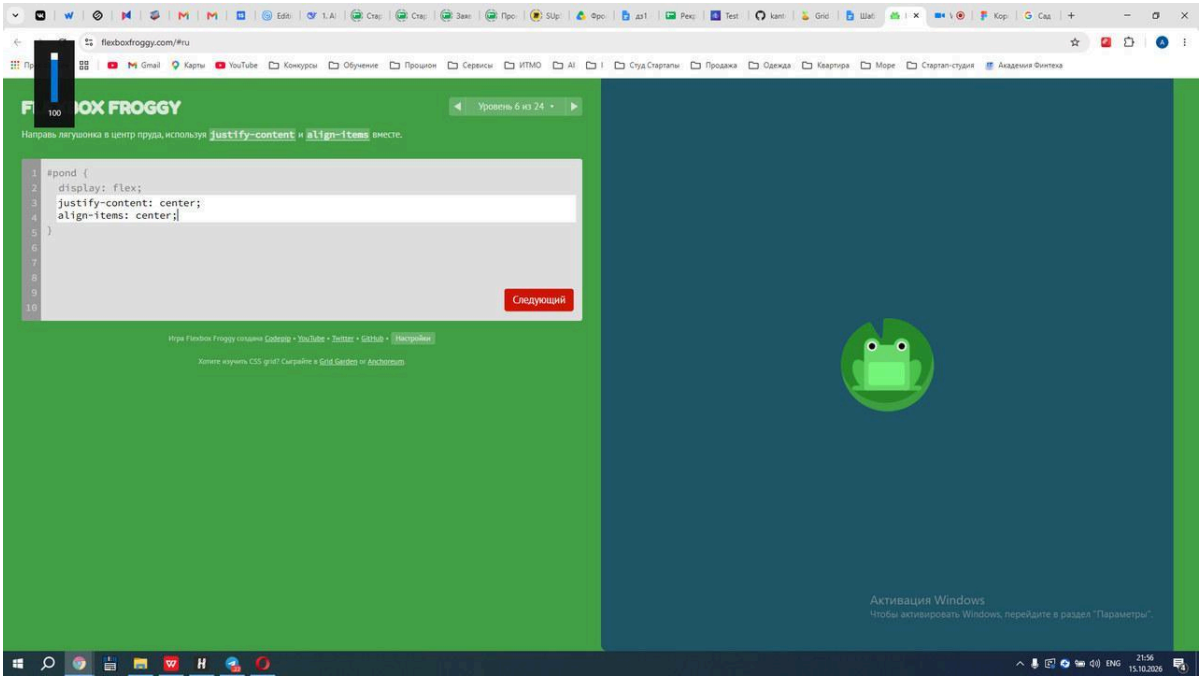


Рисунок 6 — Flexbox Froggy, уровень 6

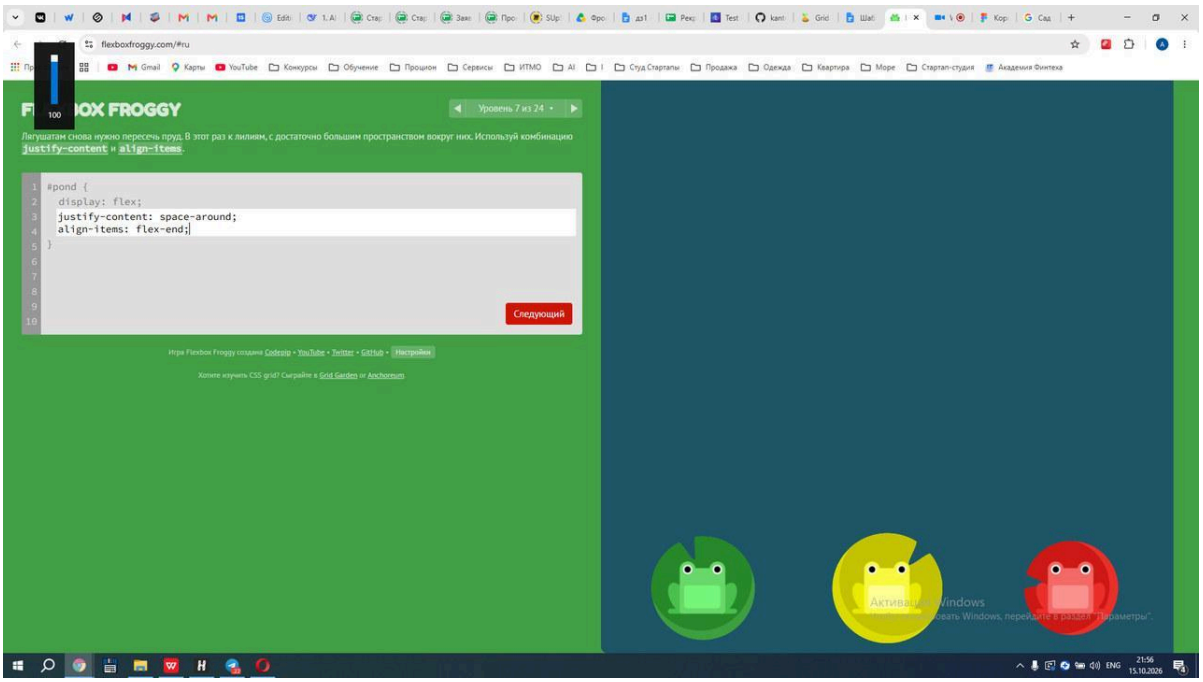


Рисунок 7 — Flexbox Froggy, уровень 7

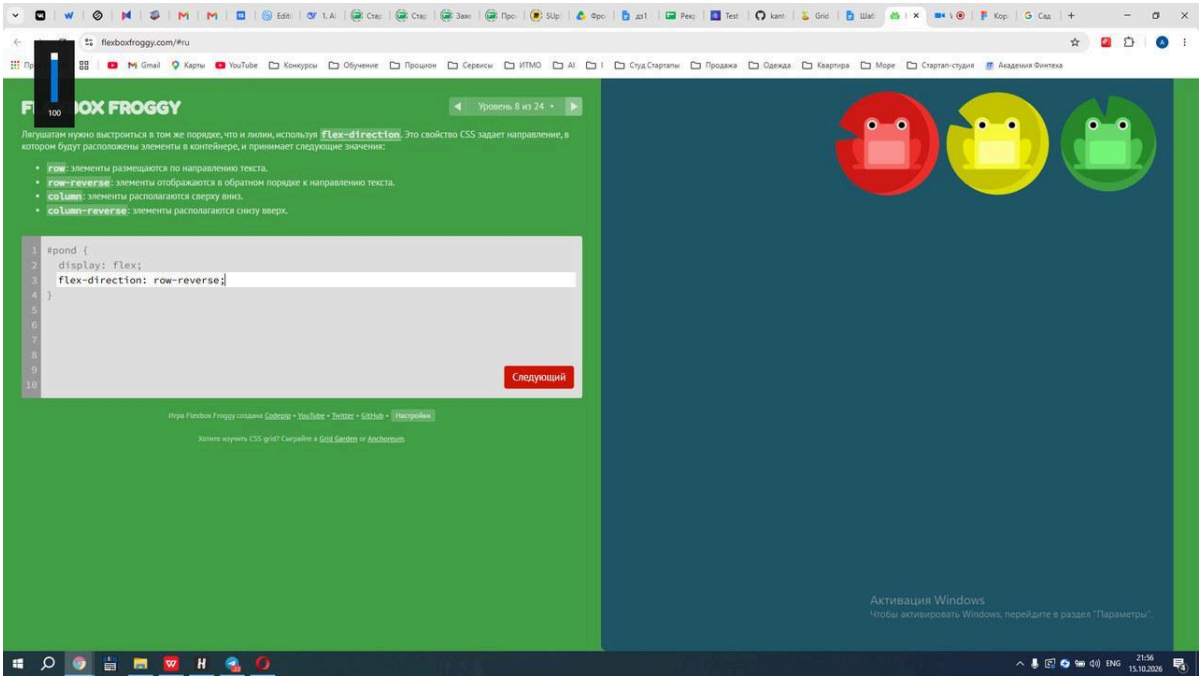


Рисунок 8 — Flexbox Froggy, уровень 8

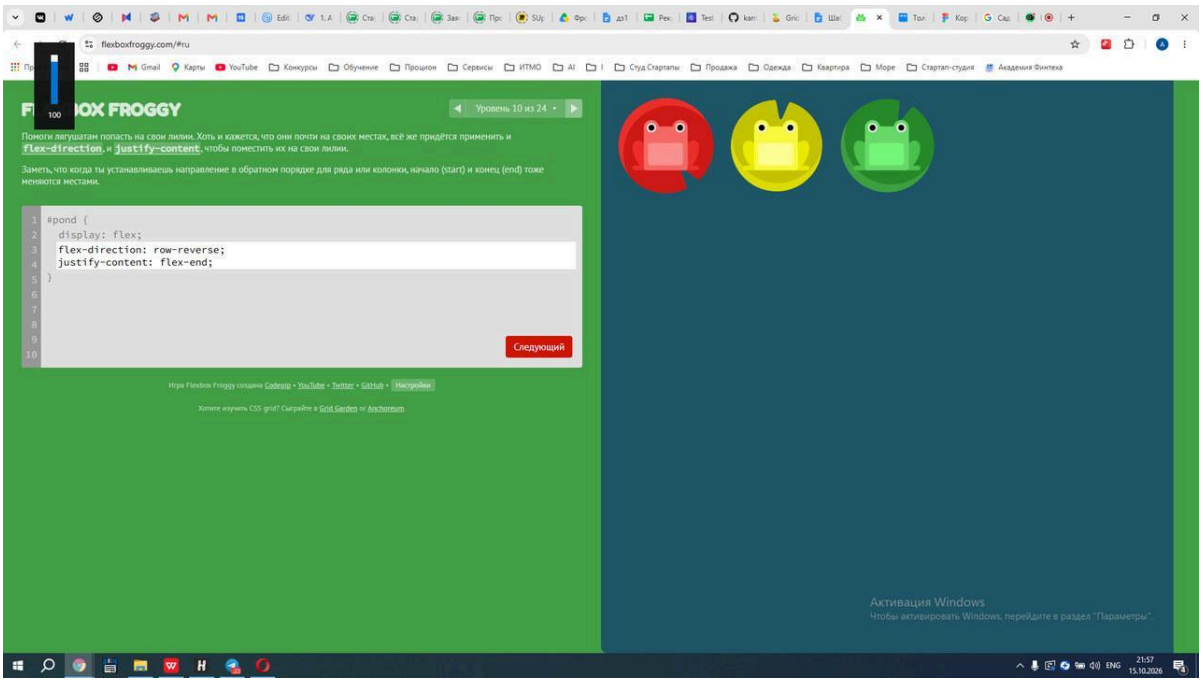


Рисунок 9 — Flexbox Froggy, уровень 9

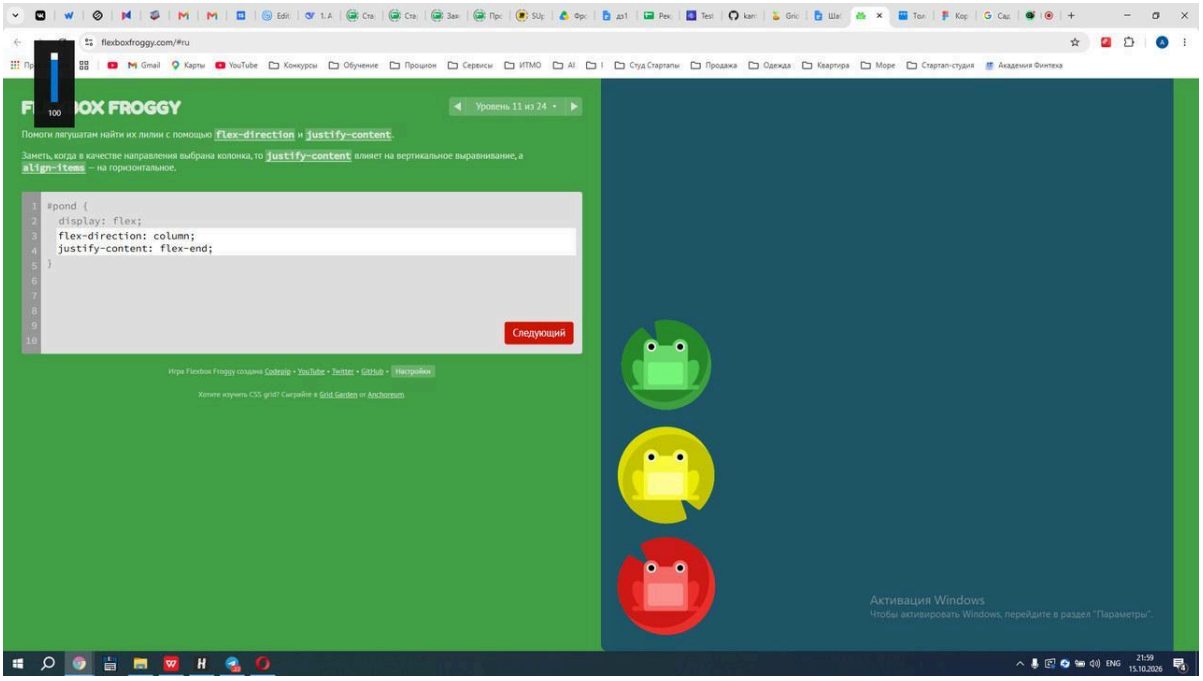


Рисунок 10 — Flexbox Froggy, уровень 10

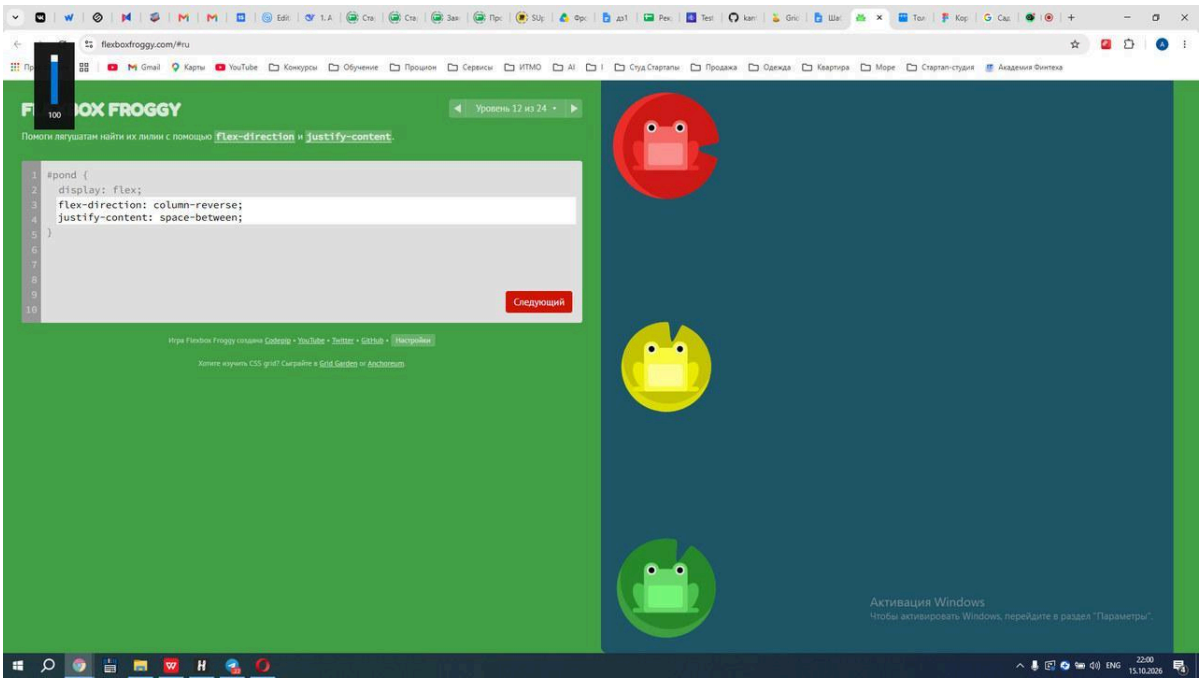


Рисунок 11 — Flexbox Froggy, уровень 11

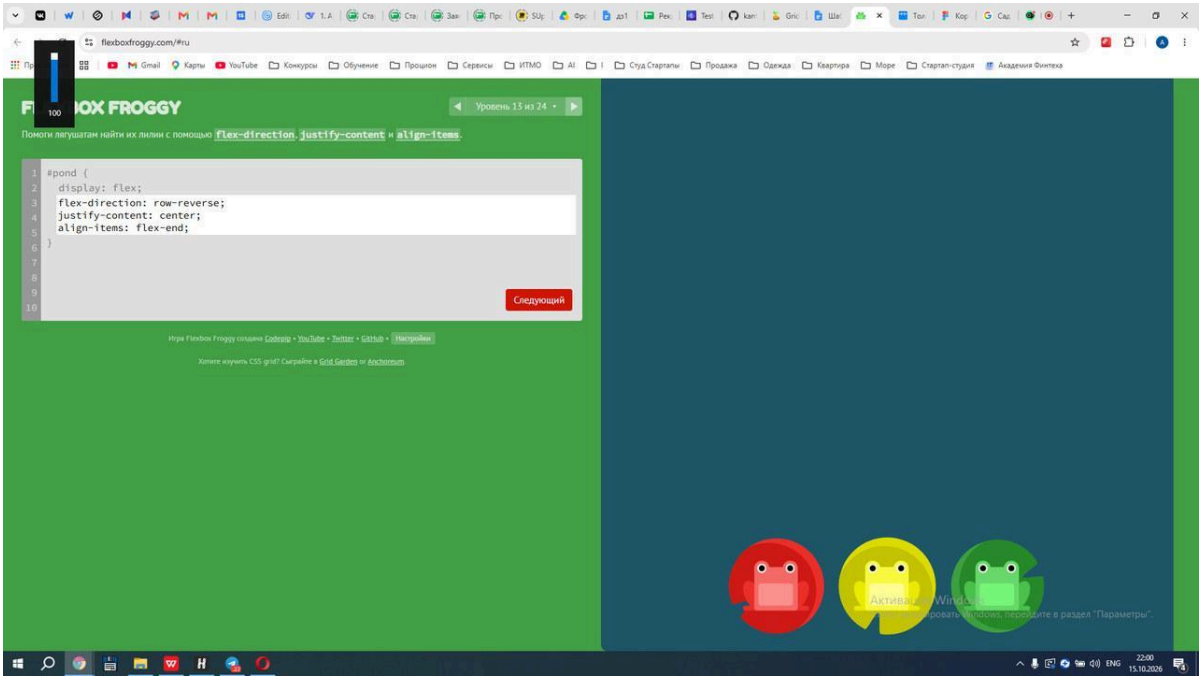


Рисунок 12 — Flexbox Froggy, уровень 12

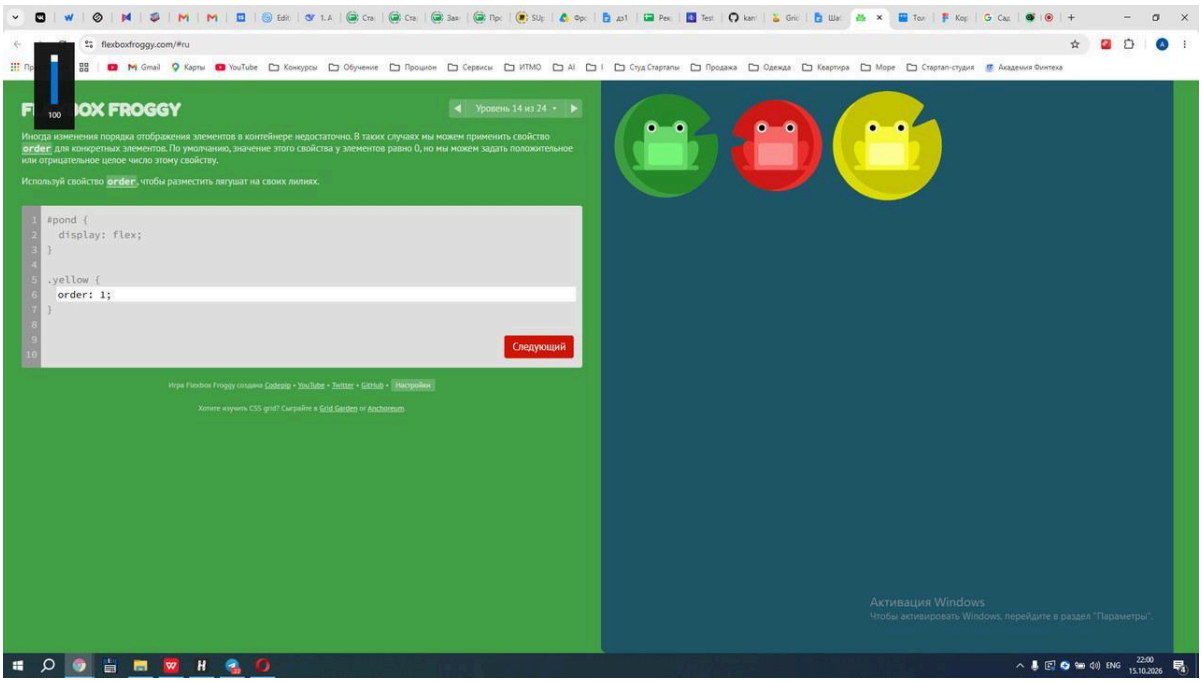


Рисунок 13 — Flexbox Froggy, уровень 13

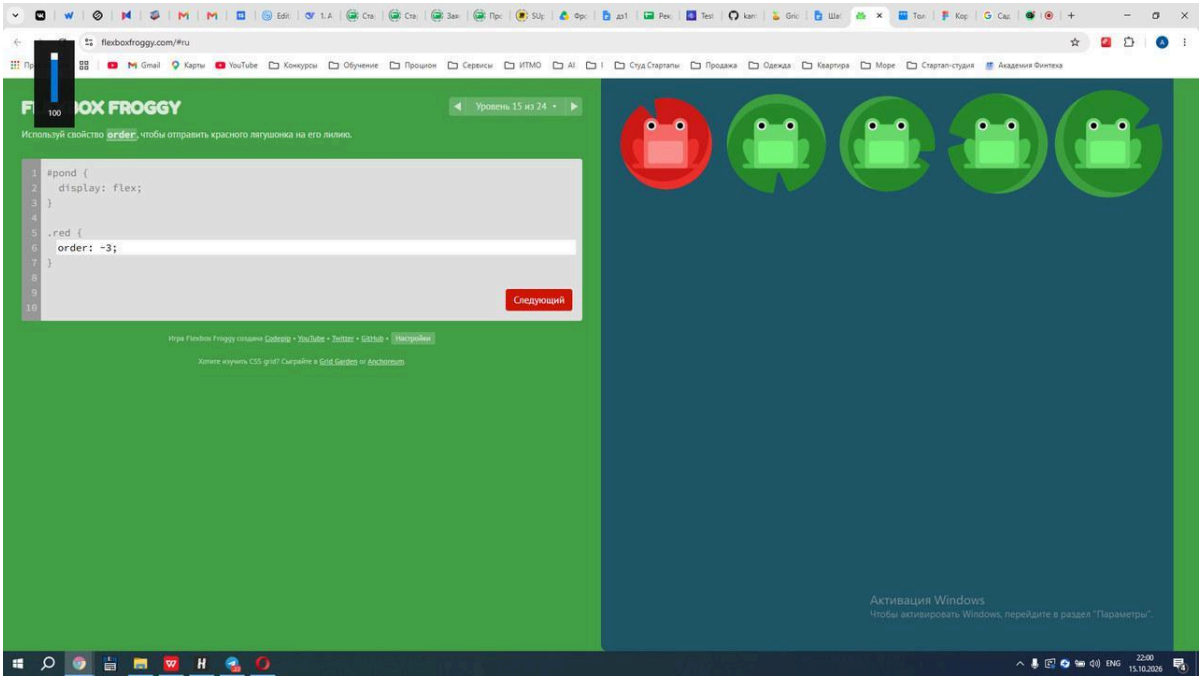


Рисунок 14 — Flexbox Froggy, уровень 14

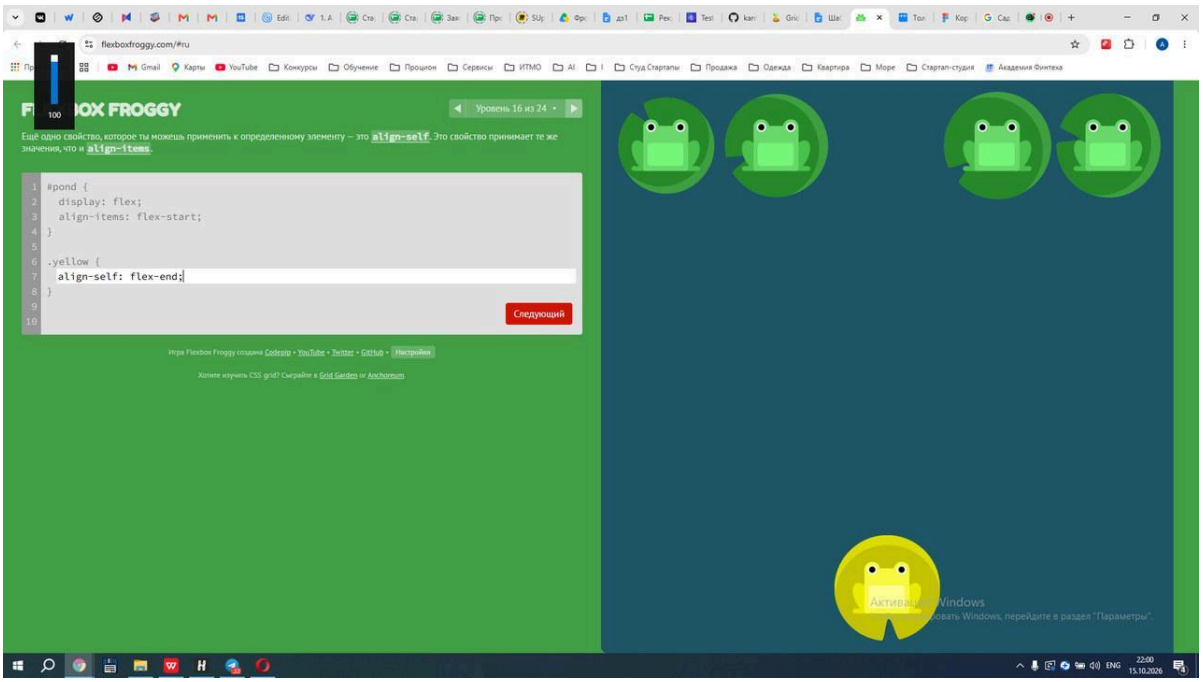


Рисунок 15 — Flexbox Froggy, уровень 15

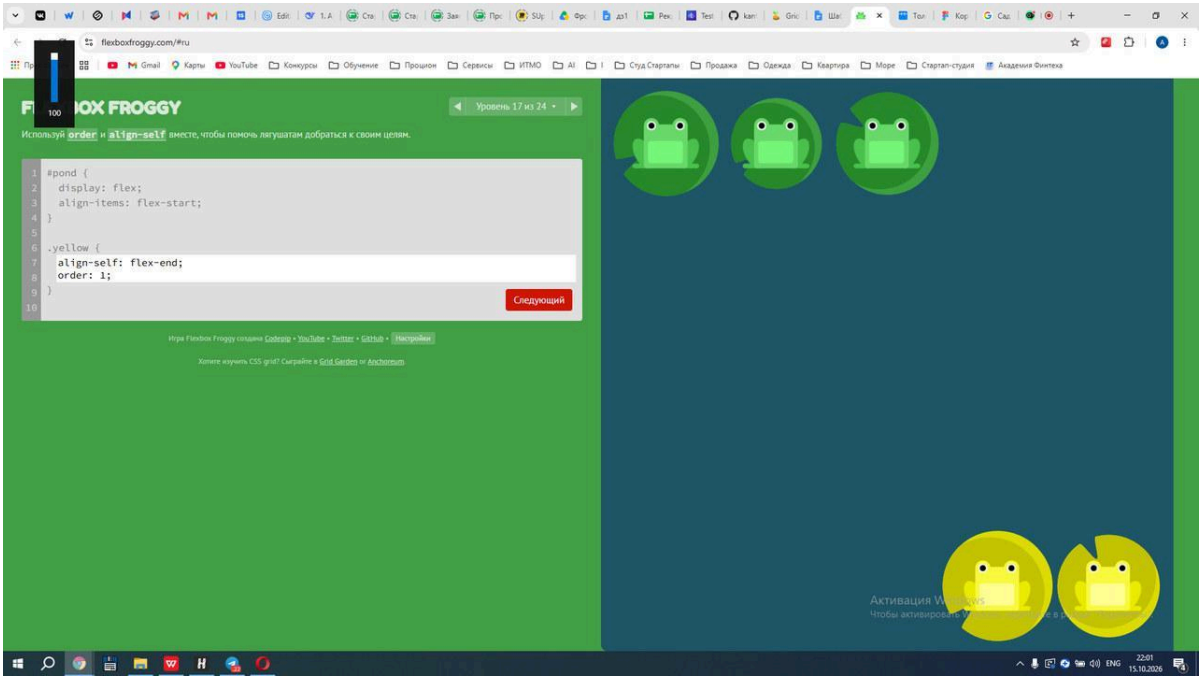


Рисунок 16 — Flexbox Froggy, уровень 16

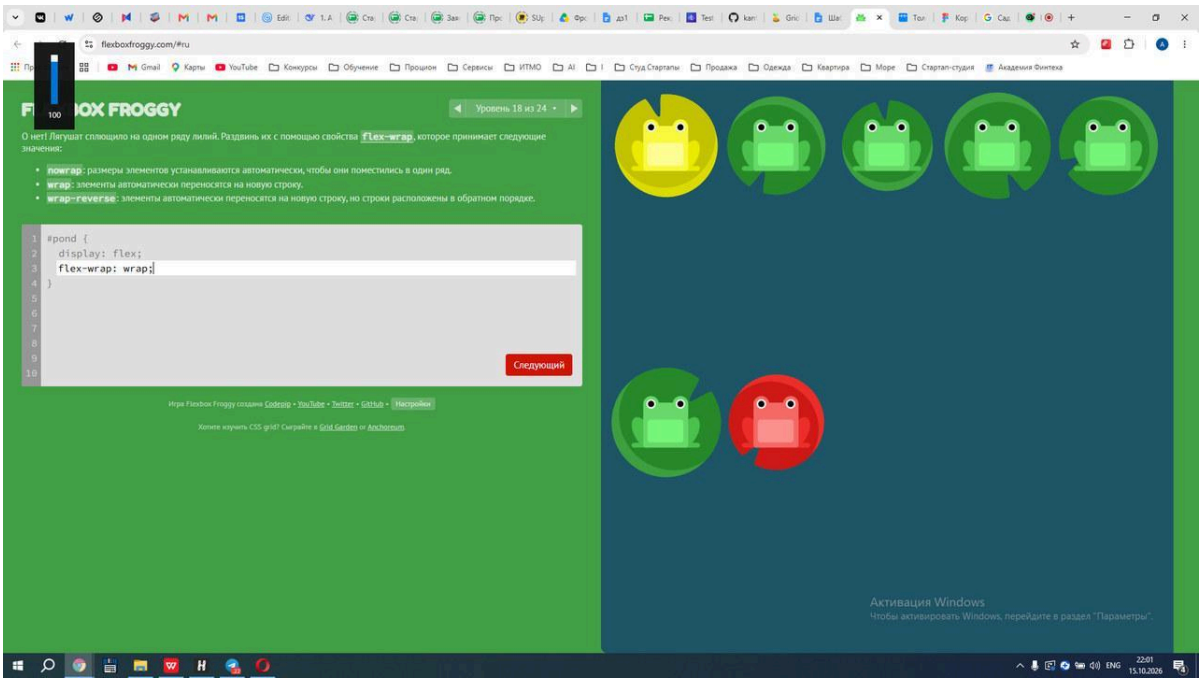


Рисунок 17 — Flexbox Froggy, уровень 17

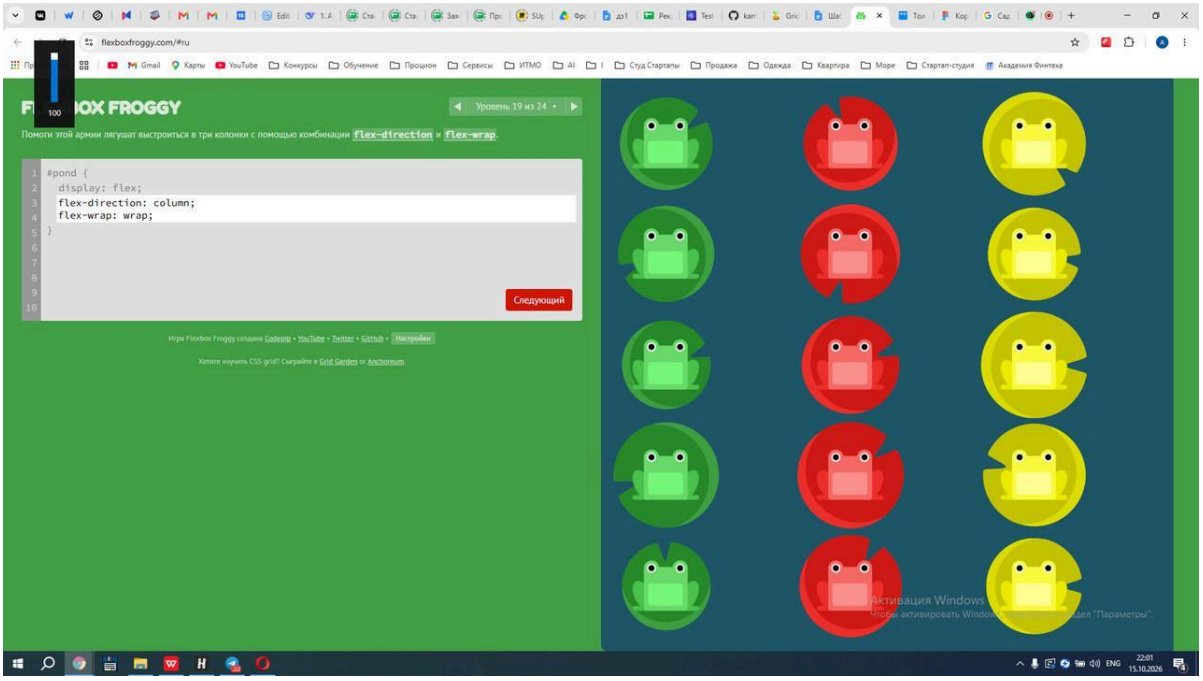


Рисунок 18 — Flexbox Froggy, уровень 18

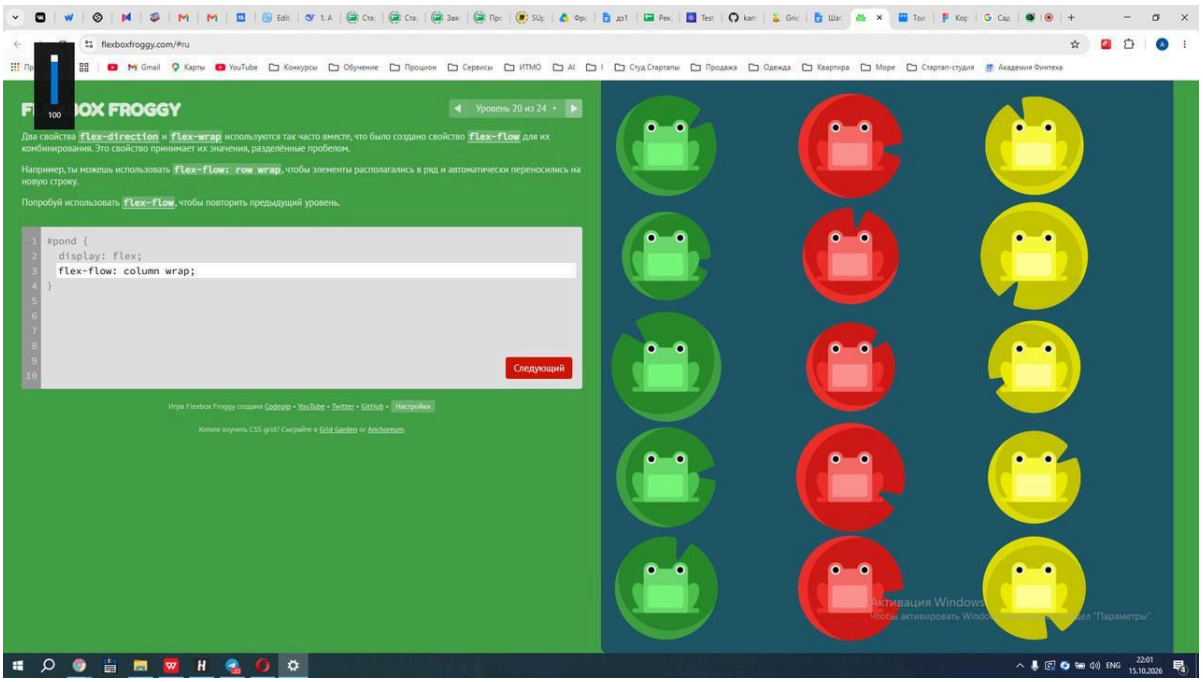


Рисунок 19 — Flexbox Froggy, уровень 19

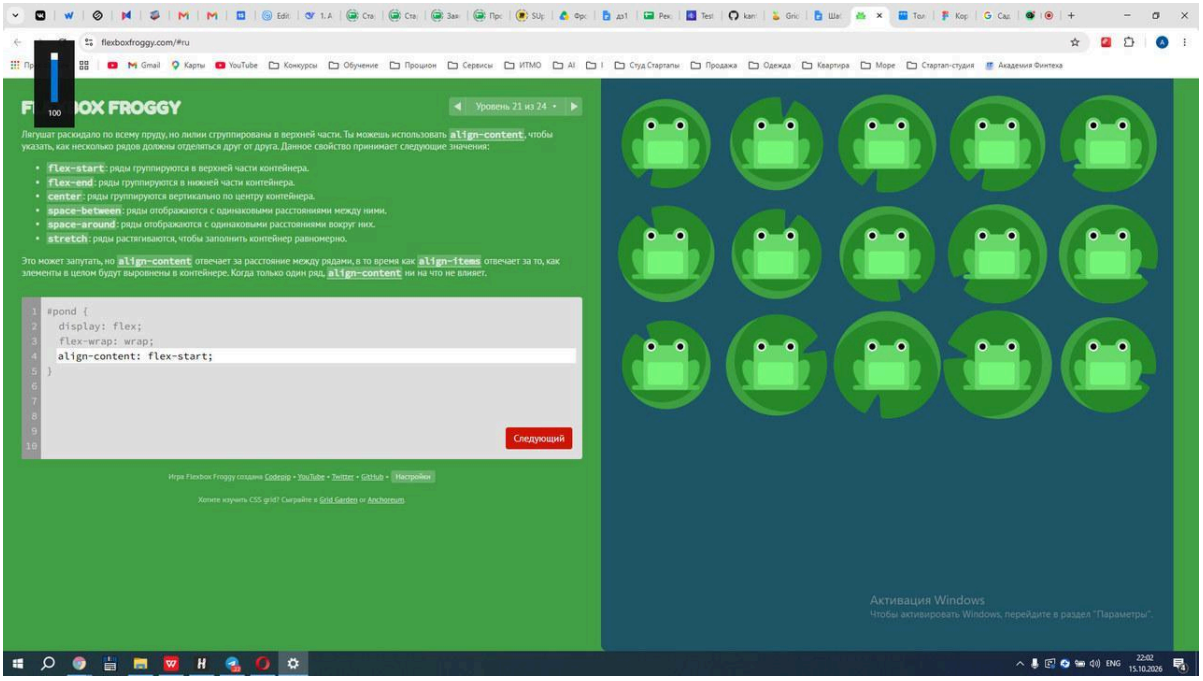


Рисунок 20 — Flexbox Froggy, уровень 20

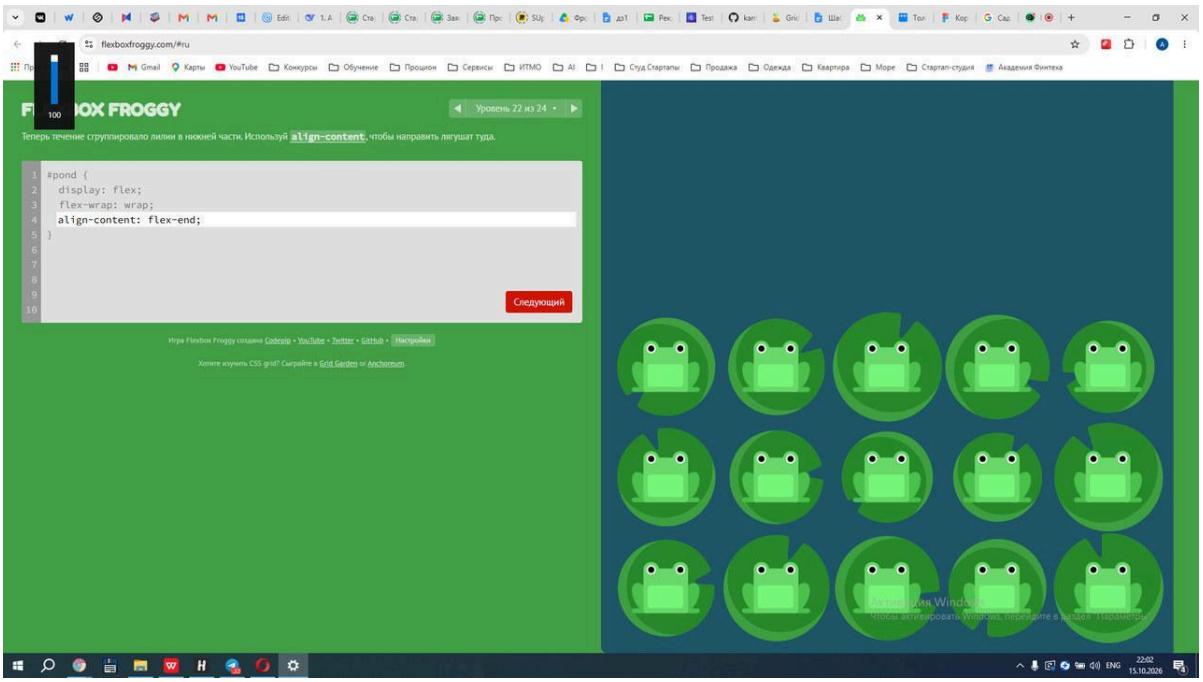


Рисунок 21 — Flexbox Froggy, уровень 21

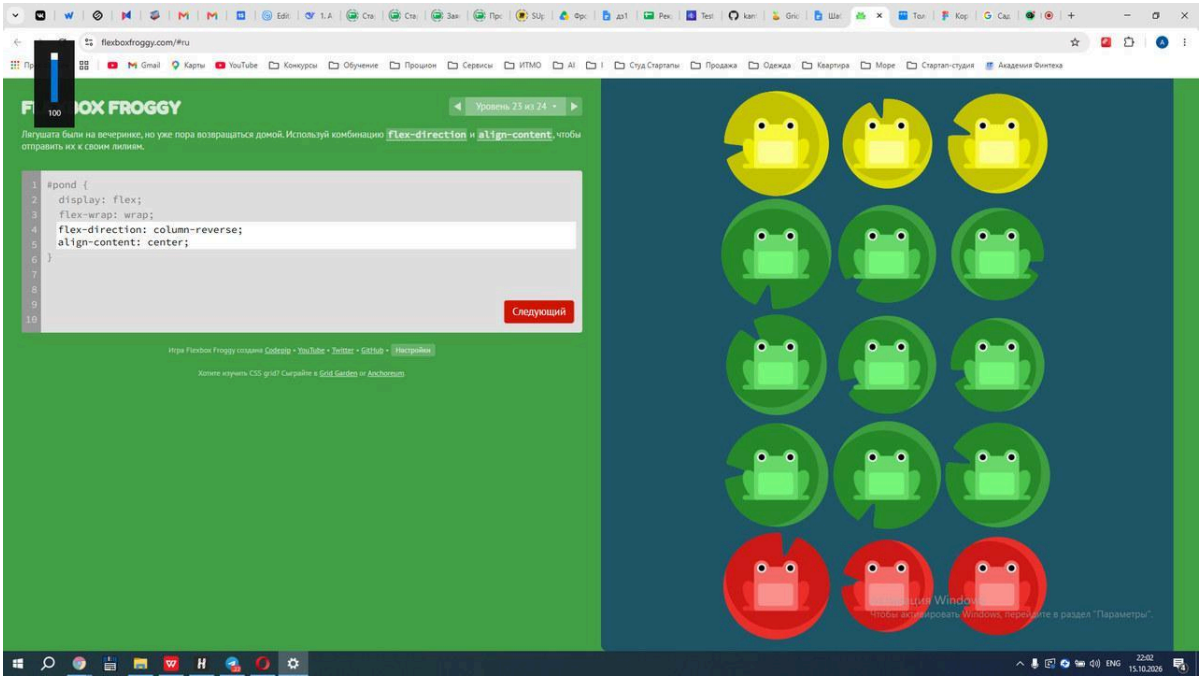


Рисунок 22 — Flexbox Froggy, уровень 22

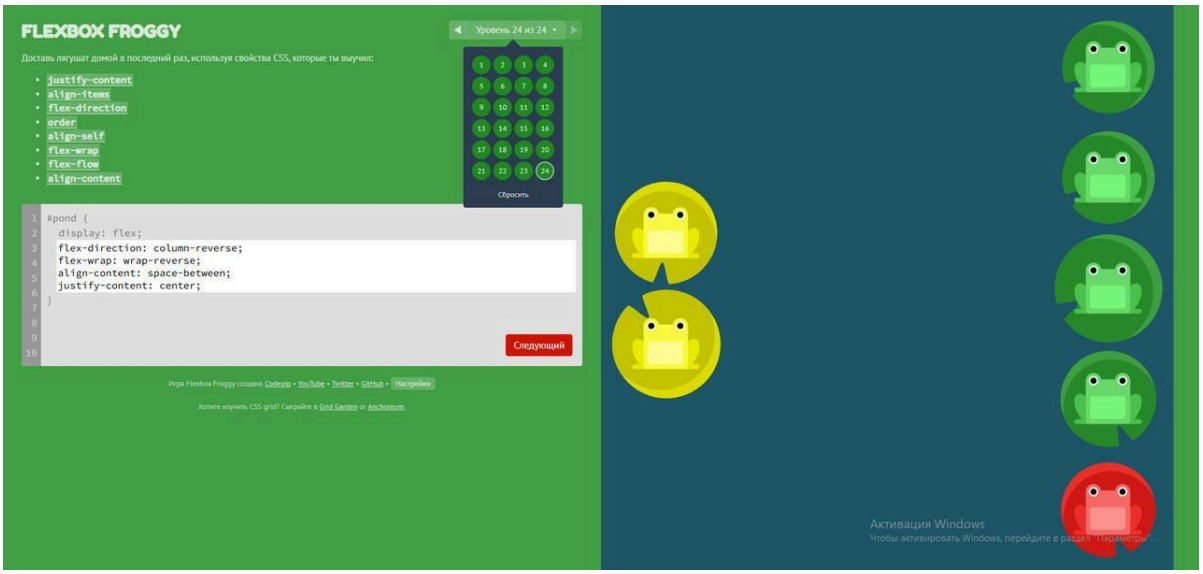


Рисунок 23 — Flexbox Froggy, уровень 23

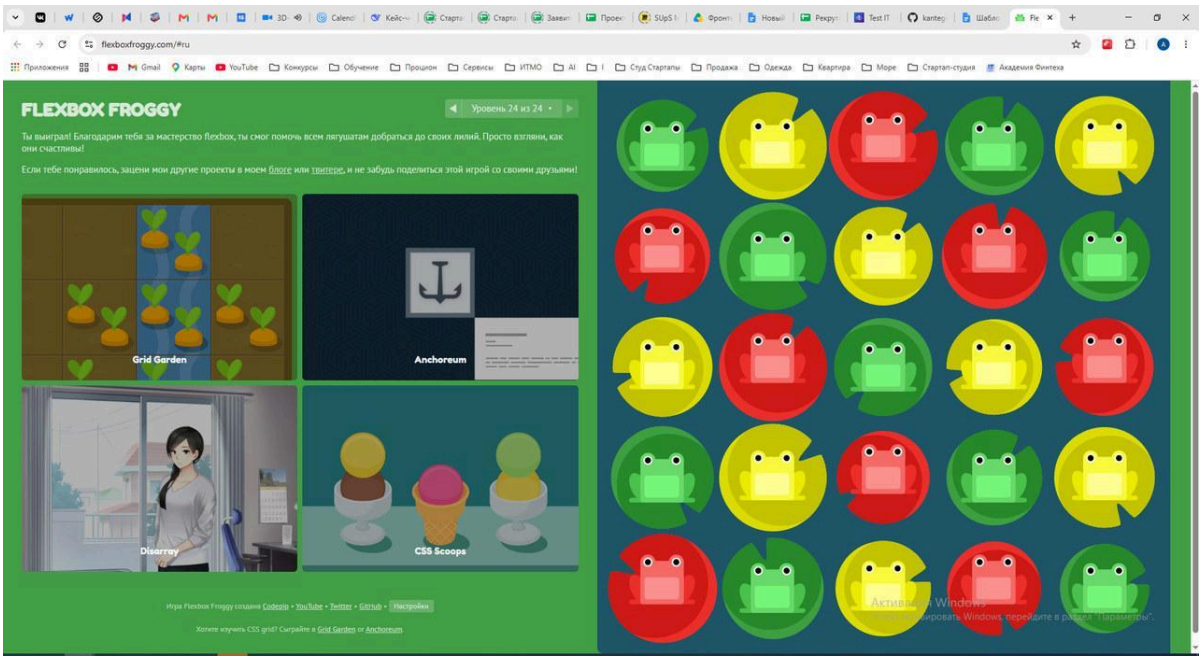


Рисунок 24 — Flexbox Froggy, уровень 24

3. CSS Grid: тренажёр Grid Garden

3.1. Описание тренажёра

CSS Grid Garden — интерактивный тренажёр по CSS Grid Layout. На поле расположены грядки с растениями и сорняками, а задача пользователя — разместить элементы в заданных ячейках или областях сетки. Для решения применяются свойства, задающие начальные и конечные линии элемента, а затем — структуру и размеры всей сетки.

В отличие от Flexbox, который прежде всего организует элементы вдоль одной оси, Grid предназначен для двумерной раскладки: строки и столбцы задаются одновременно. Это позволяет управлять как положением элемента, так и размером столбцов и строк.

3.2. Изученные свойства CSS Grid

Свойство / запись	**Назначение и пояснение**
grid-column-start	Задаёт вертикальную линию сетки, с которой начинается элемент (линию начала столбца).
grid-column-end	Задаёт линию, на которой заканчивается размещение элемента; конечная линия не включается в занимаемый диапазон.
span N	Ключевое слово span задаёт длину размещения на N столбцов или строк, а не конкретный номер конечной линии.
grid-column	Краткая запись начала и конца размещения по столбцам, например grid-column: 2 / 5.
grid-row-start / grid-row	Определяет начало или диапазон размещения элемента по строкам сетки.
grid-area	Позволяет указать область через четыре линии: row-start / column-start / row-end / column-end.
grid-template-columns	Задаёт количество и размеры столбцов сетки. В качестве размеров могут использоваться %, px, em и fr.
grid-template-rows	Задаёт количество и размеры строк сетки.
fr	Доля свободного пространства в Grid-контейнере. Например, 1fr 5fr делит доступное пространство в отношении 1 к 5.
grid-template	Краткая запись шаблона строк и столбцов: сначала значения строк, затем через косую черту — столбцов.

3.3. Темы уровней и результат прохождения

Уровни	Основной изучаемый приём
1–9	Положение элементов по столбцам: начало, конец, отрицательные линии и span.
10–11	Сокращённая запись grid-column.
12–15	Положение элементов по строкам и совместное задание диапазонов строк и столбцов.
16–18	Размещение элемента по четырём линиям с помощью grid-area.
19–24	Определение количества и размеров столбцов/строк через проценты, абсолютные единицы и fr.
25–28	Комплексное задание шаблона строк и столбцов с помощью grid-template.

Результат: в предоставленных материалах присутствуют скриншоты всех 28 уровней Grid Garden и отдельный итоговый экран завершения. Таким образом, в отчёте показана последовательность прохождения от базового позиционирования до построения сетки с заданными размерами строк и столбцов.

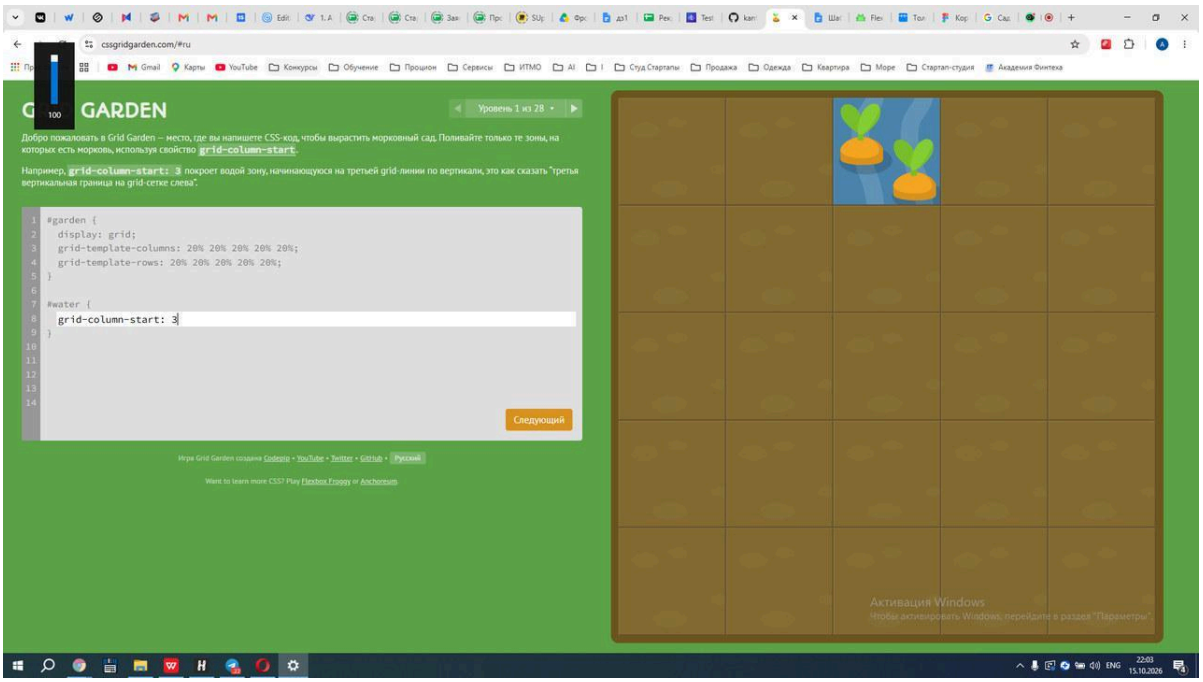


Рисунок 25 — CSS Grid Garden, уровень 1

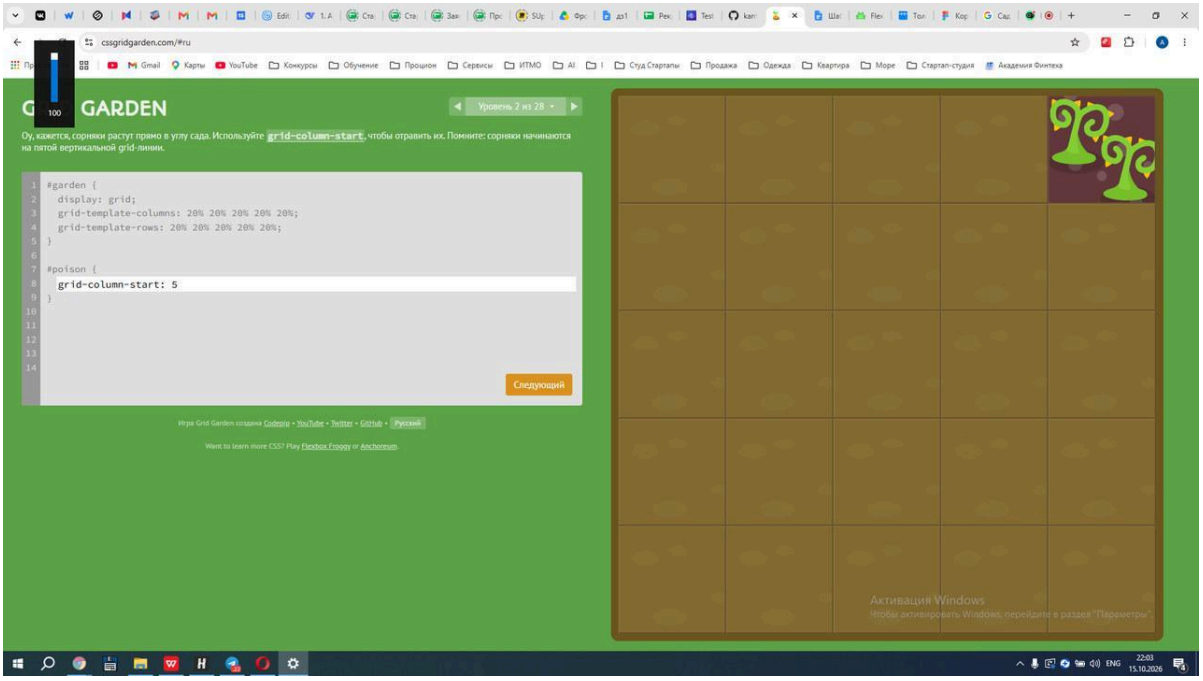


Рисунок 26 — CSS Grid Garden, уровень 2

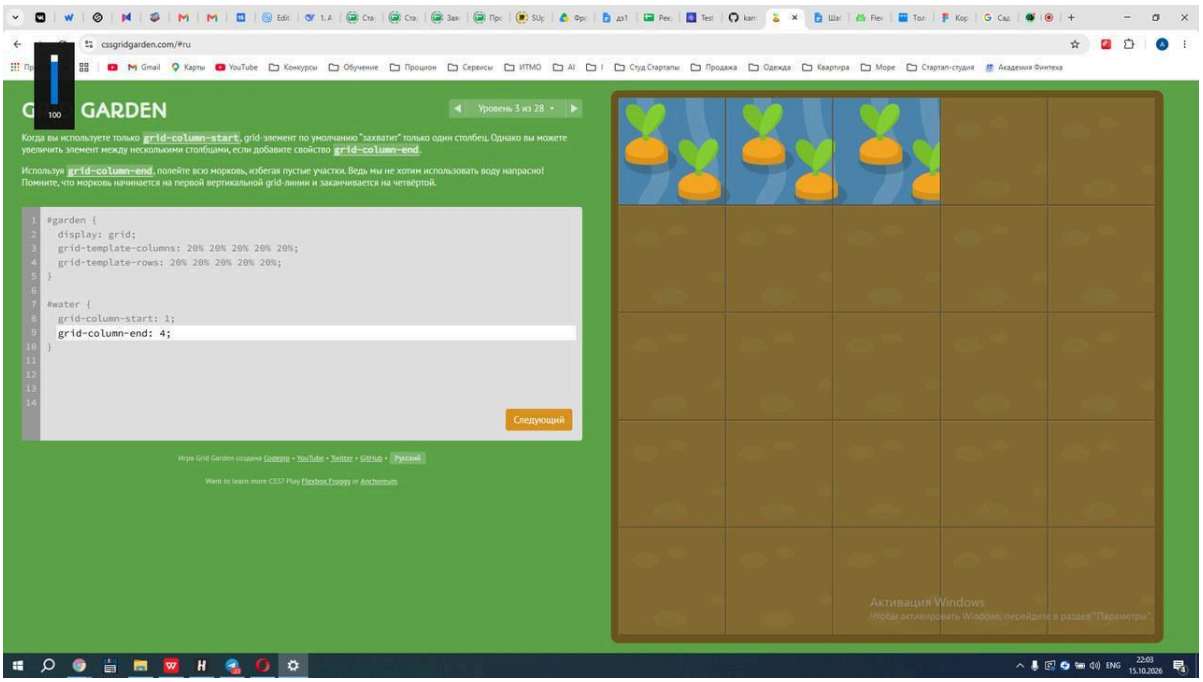


Рисунок 27 — CSS Grid Garden, уровень 3

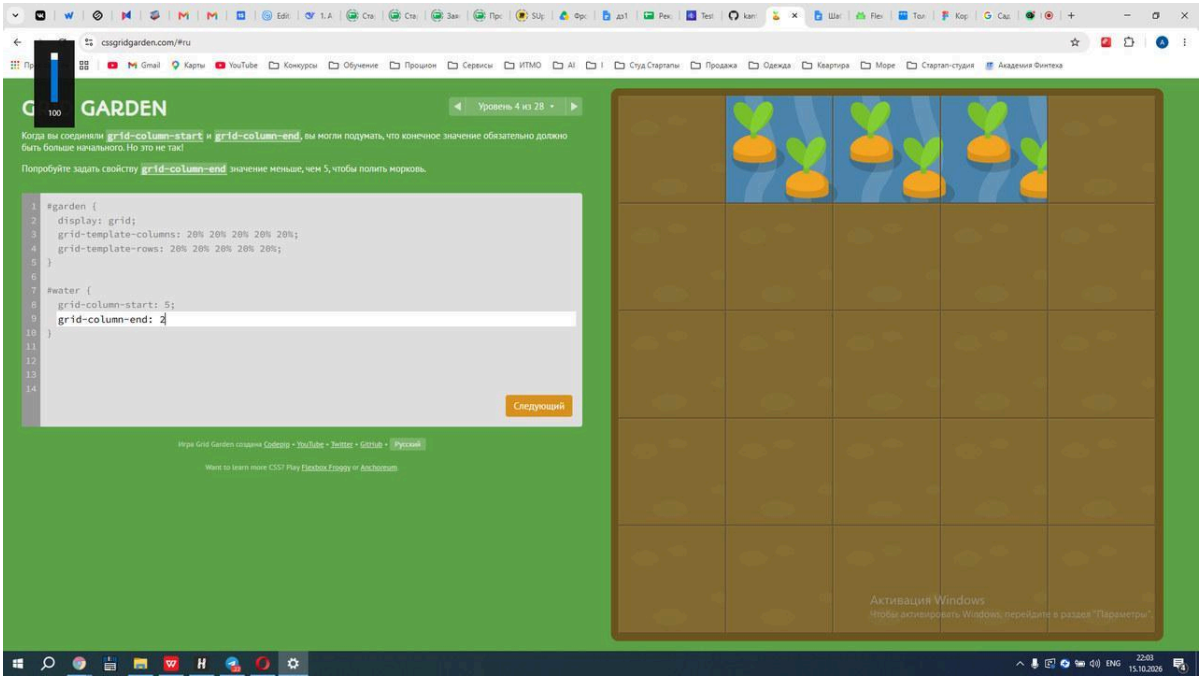


Рисунок 28 — CSS Grid Garden, уровень 4

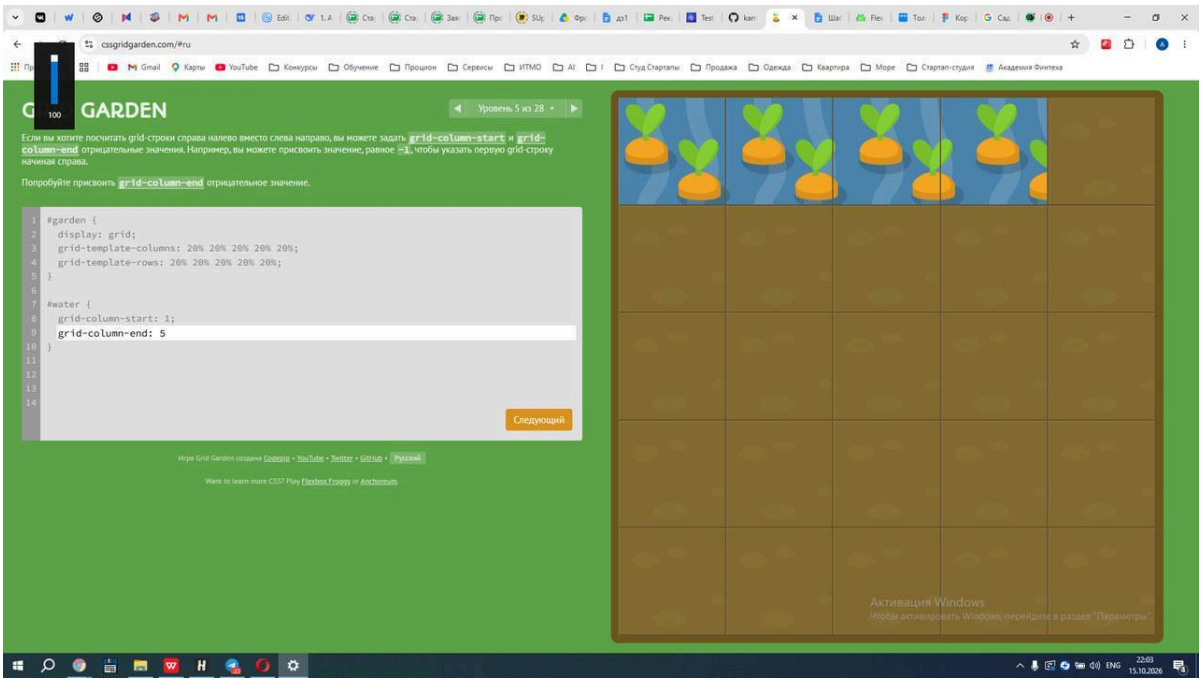


Рисунок 29 — CSS Grid Garden, уровень 5

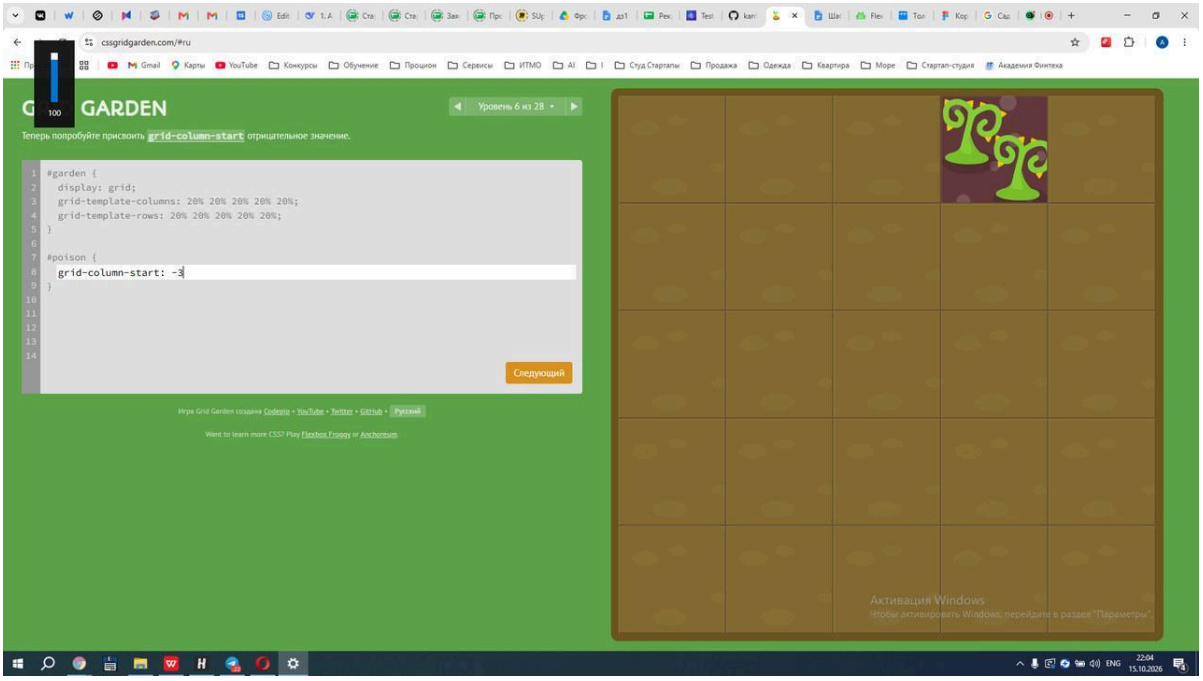


Рисунок 30 — CSS Grid Garden, уровень 6

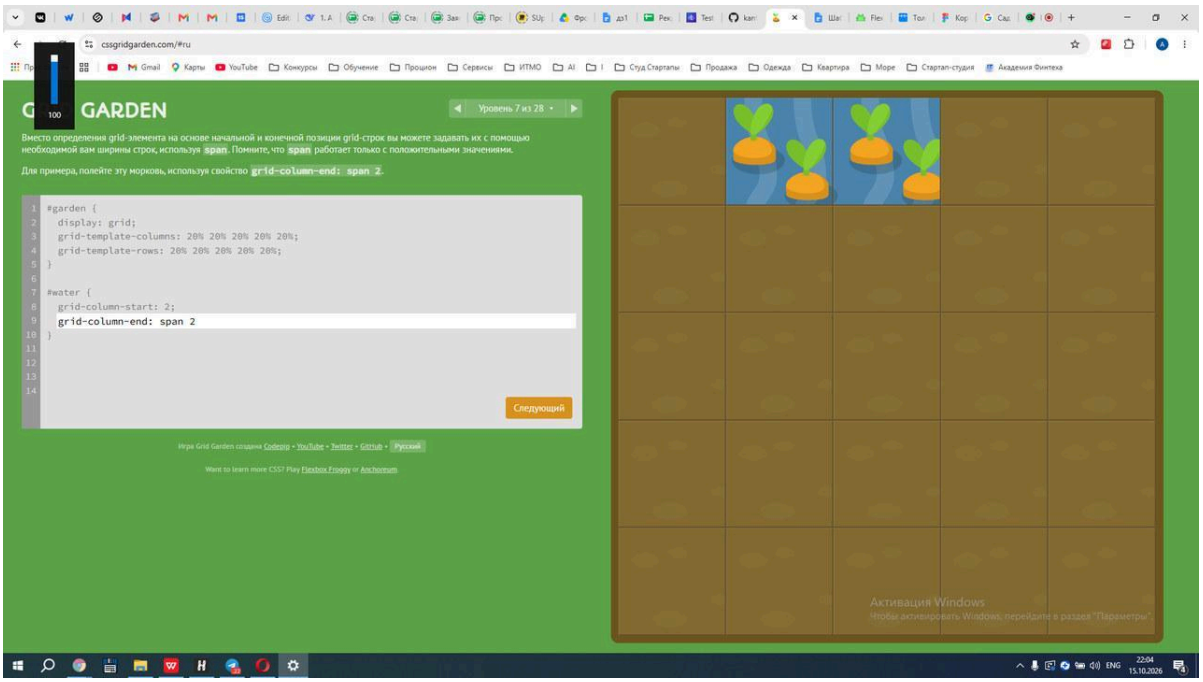


Рисунок 31 — CSS Grid Garden, уровень 7

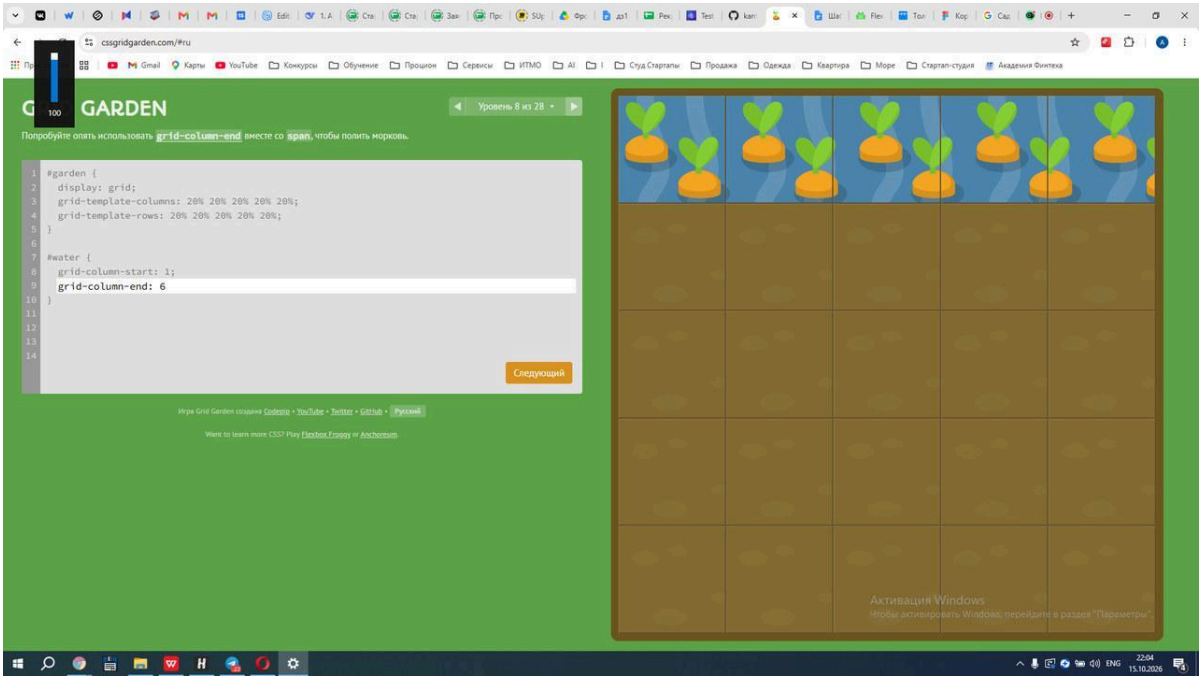


Рисунок 32 — CSS Grid Garden, уровень 8

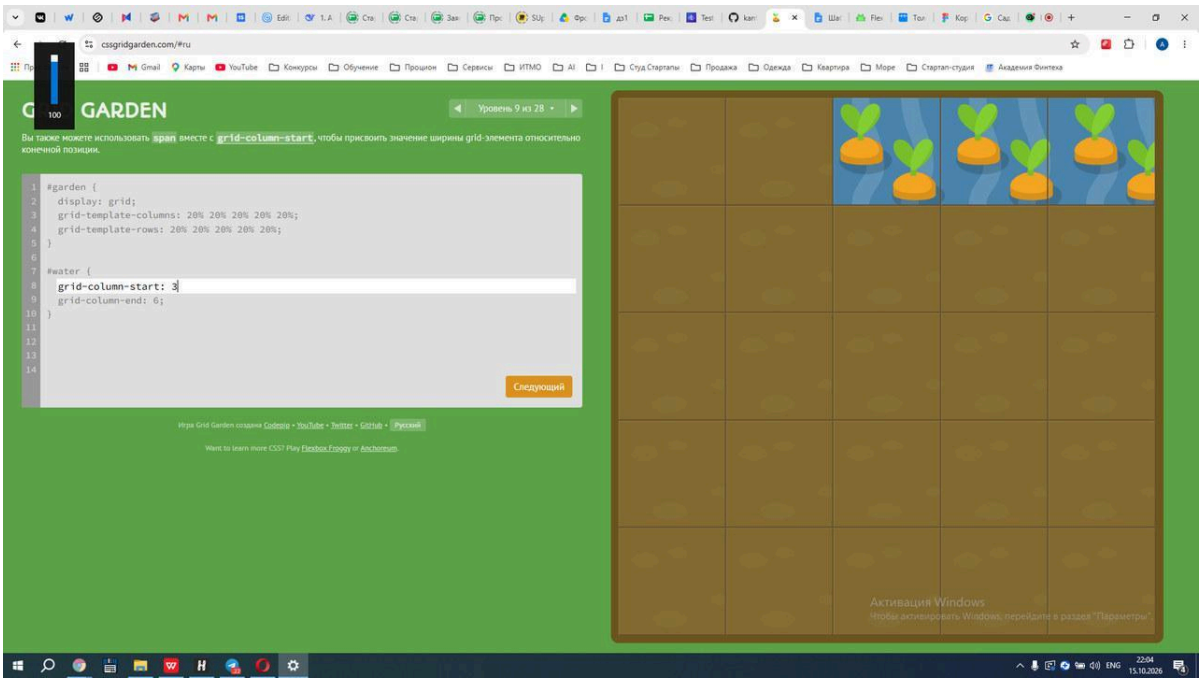


Рисунок 33 — CSS Grid Garden, уровень 9

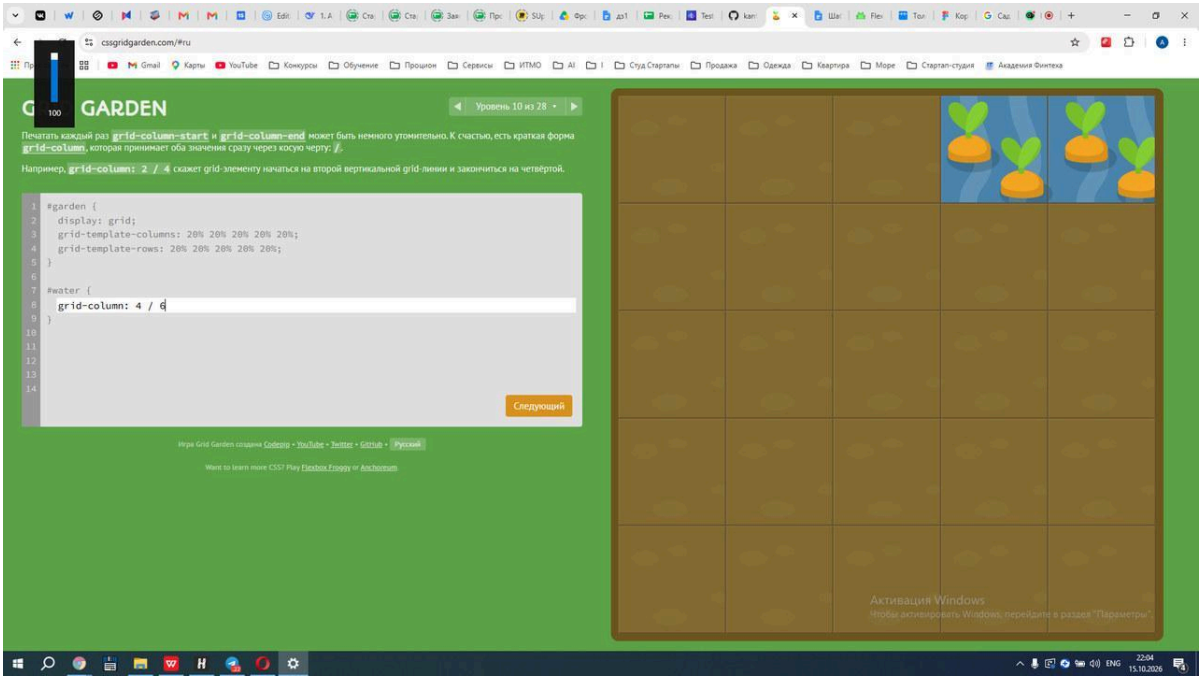


Рисунок 34 — CSS Grid Garden, уровень 10

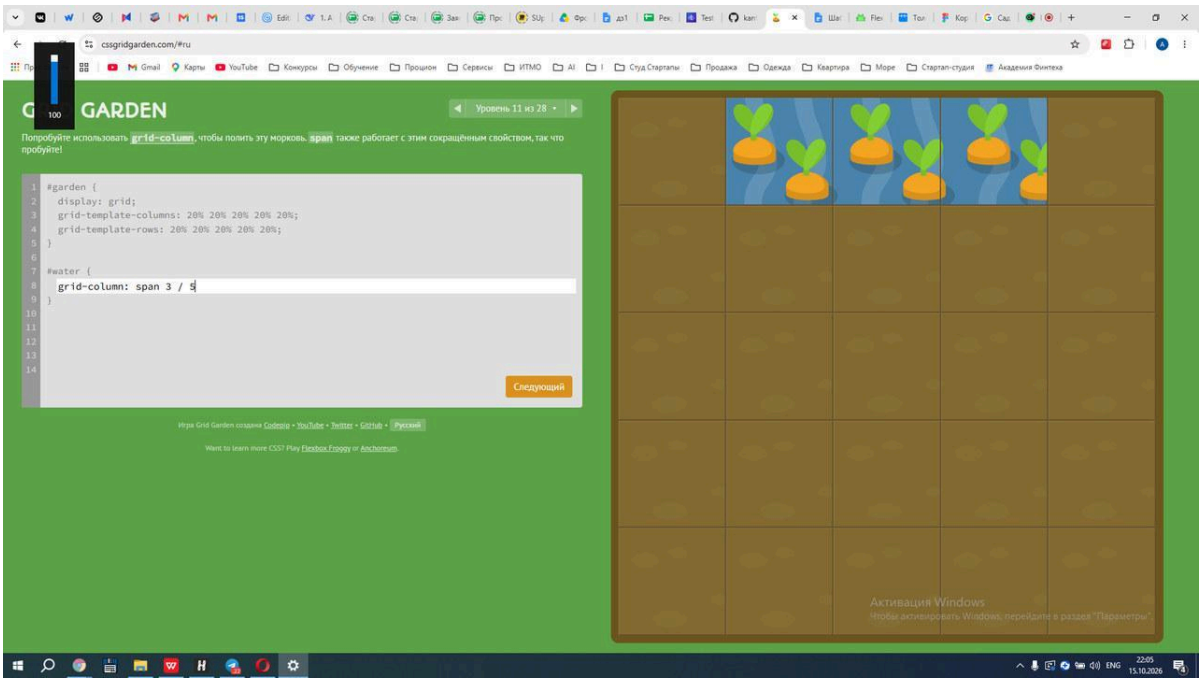


Рисунок 35 — CSS Grid Garden, уровень 11

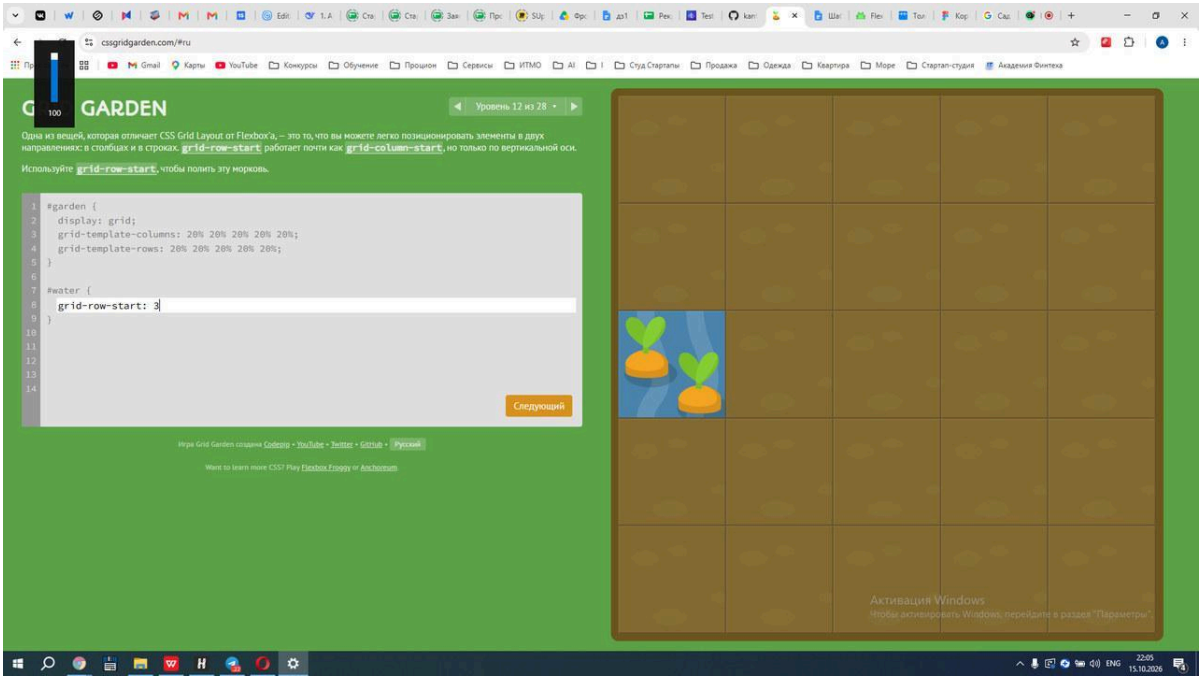


Рисунок 36 — CSS Grid Garden, уровень 12

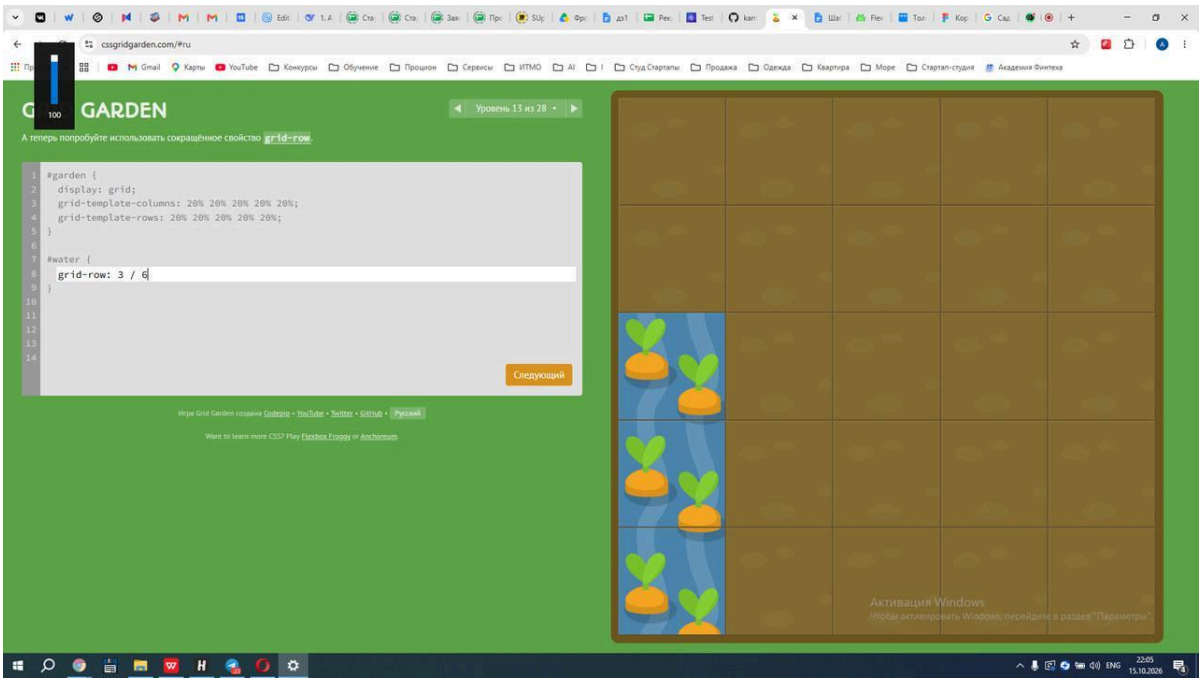


Рисунок 37 — CSS Grid Garden, уровень 13

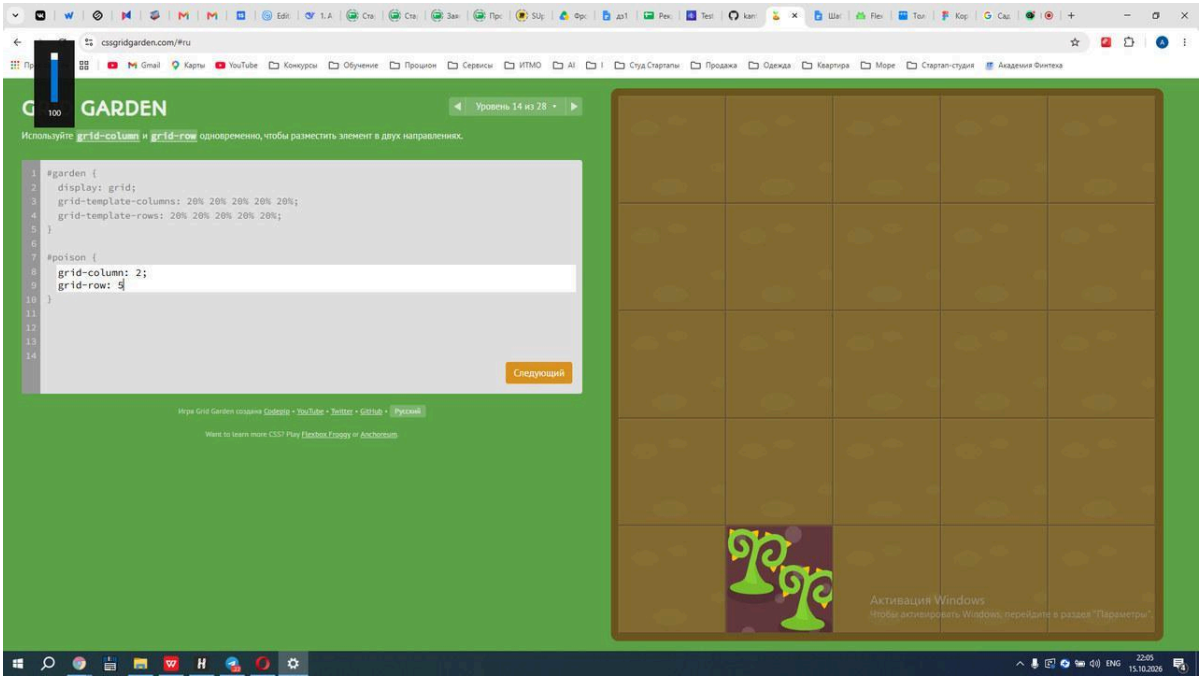


Рисунок 38 — CSS Grid Garden, уровень 14

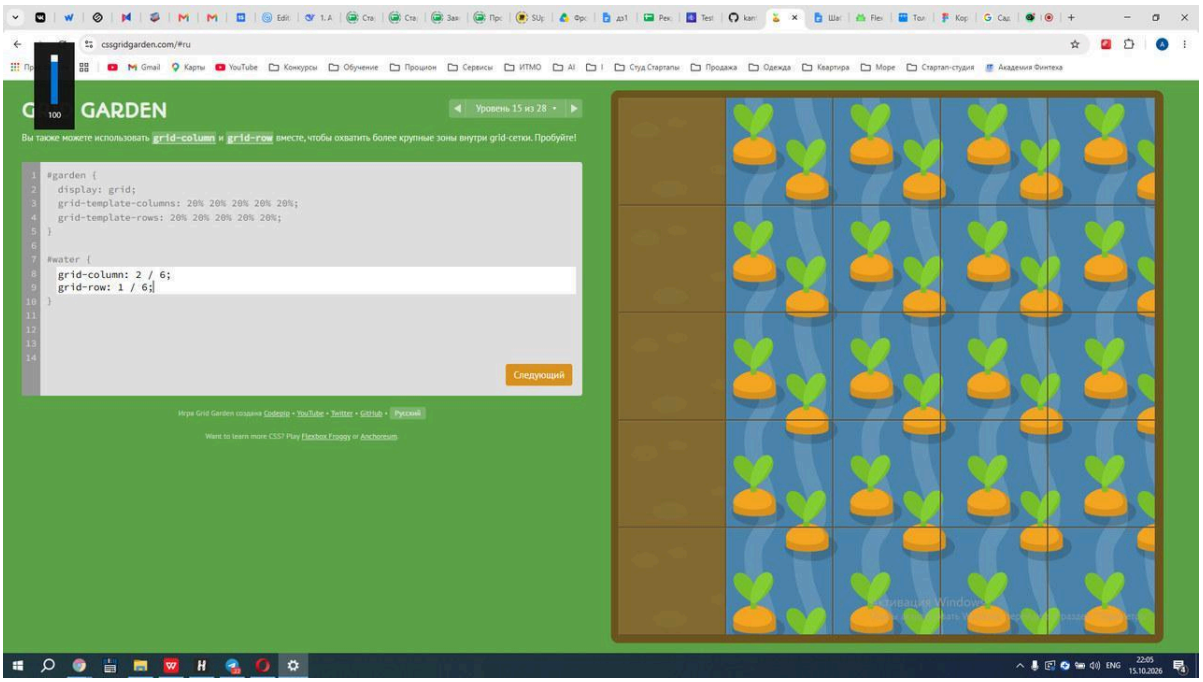


Рисунок 39 — CSS Grid Garden, уровень 15

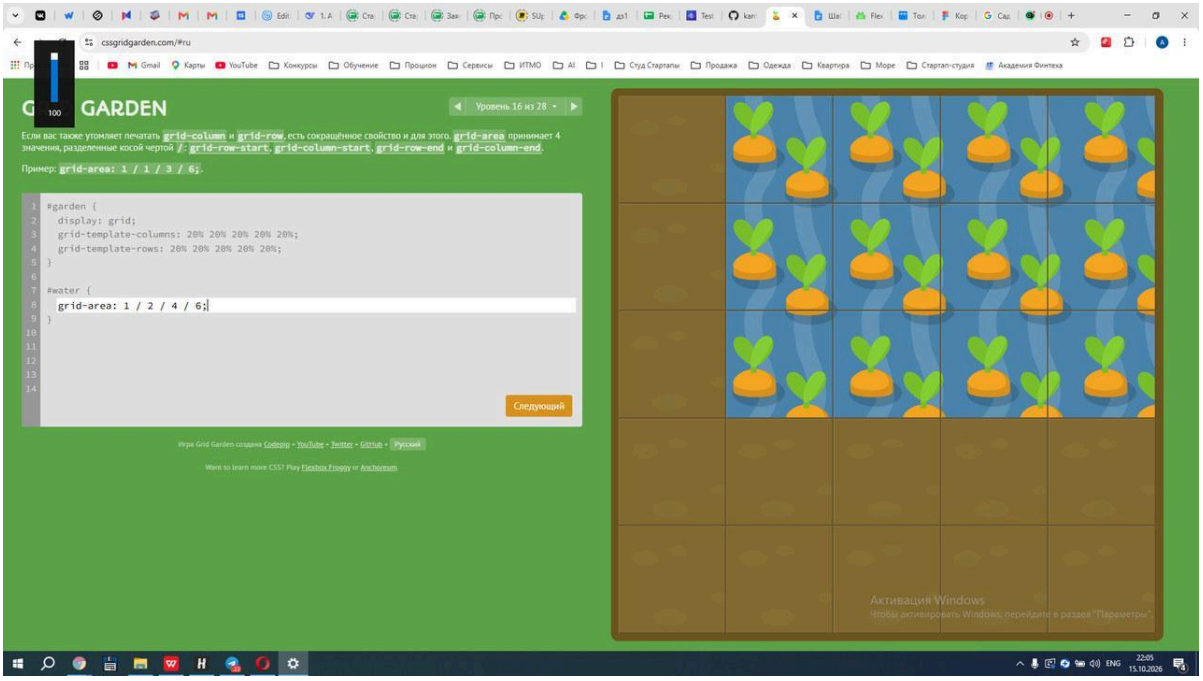


Рисунок 40 — CSS Grid Garden, уровень 16

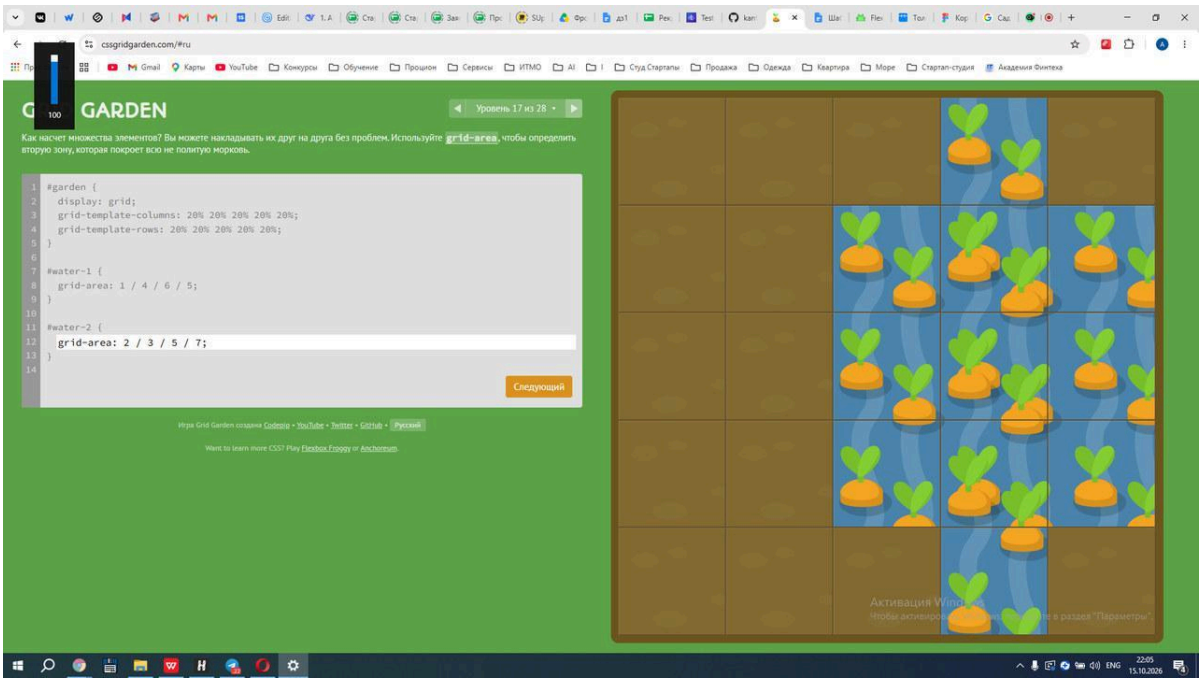


Рисунок 41 — CSS Grid Garden, уровень 17

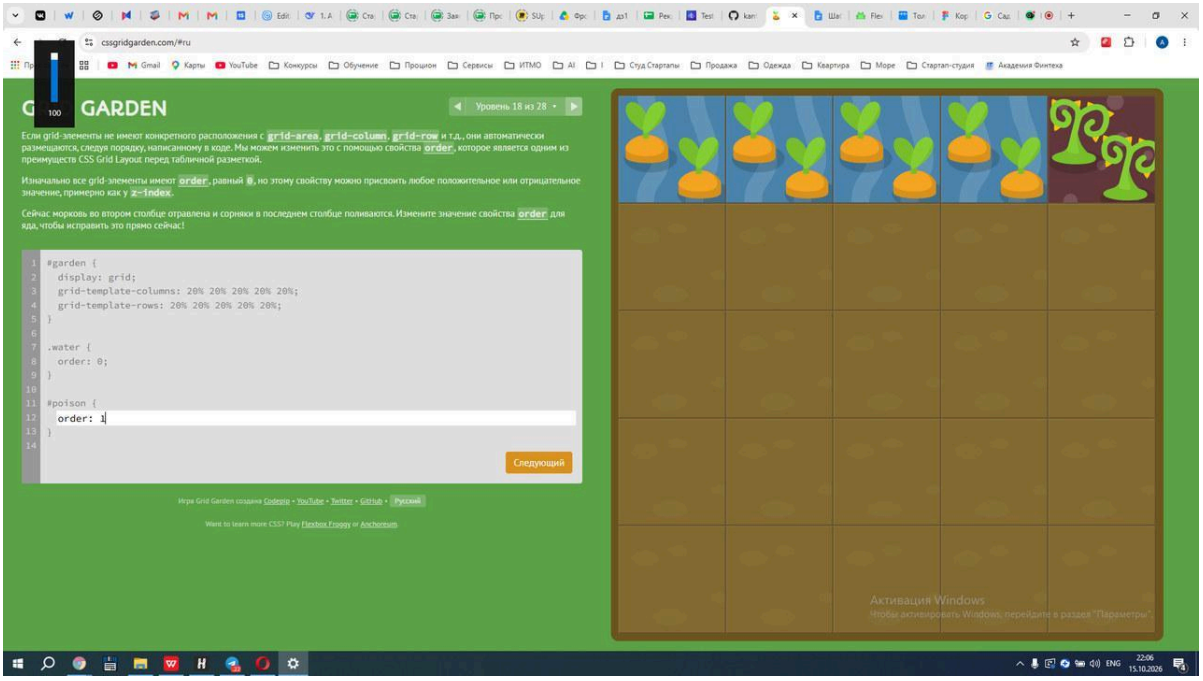


Рисунок 42 — CSS Grid Garden, уровень 18

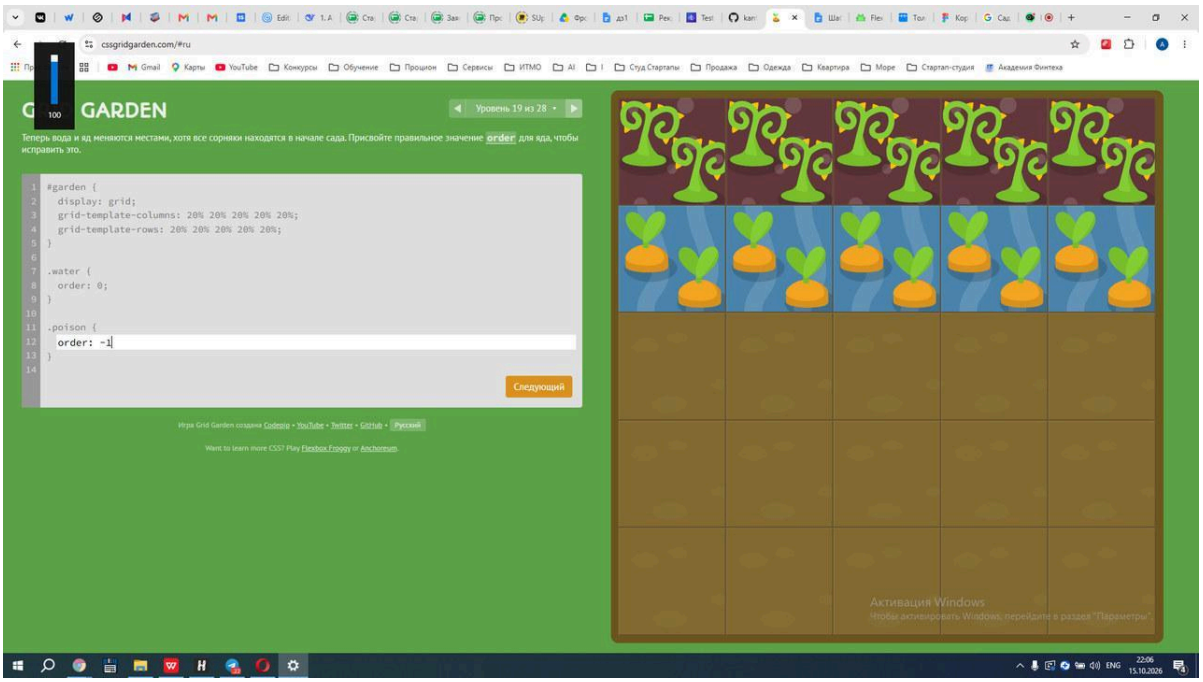


Рисунок 43 — CSS Grid Garden, уровень 19

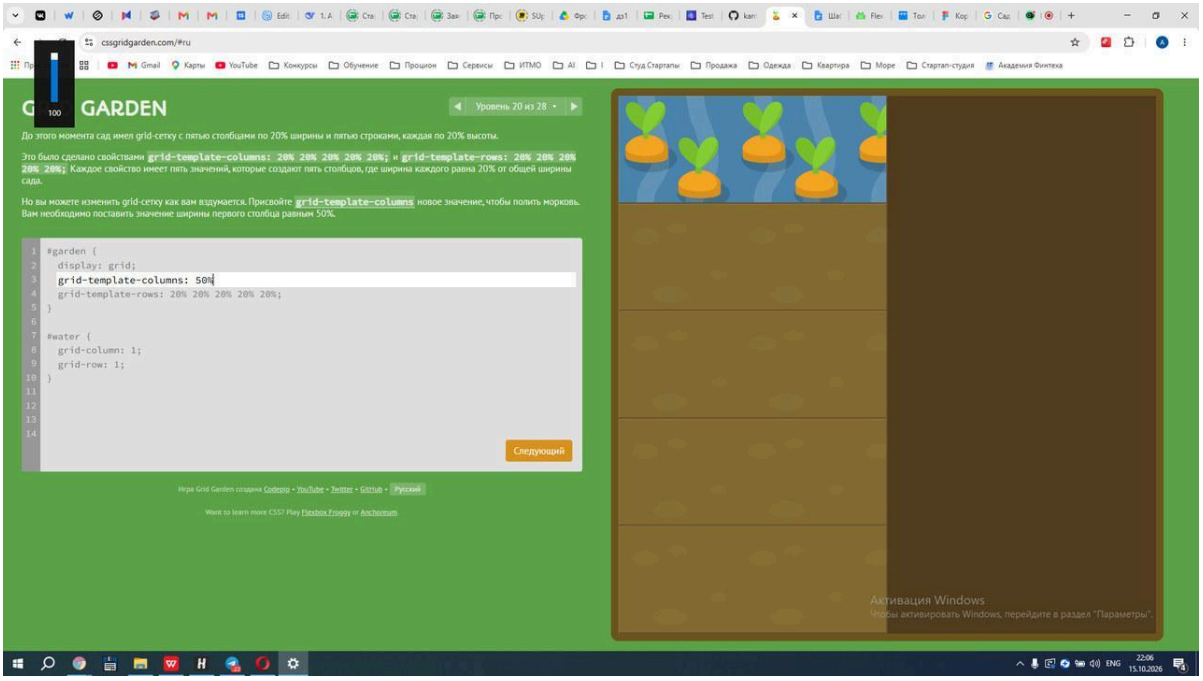


Рисунок 44 — CSS Grid Garden, уровень 20

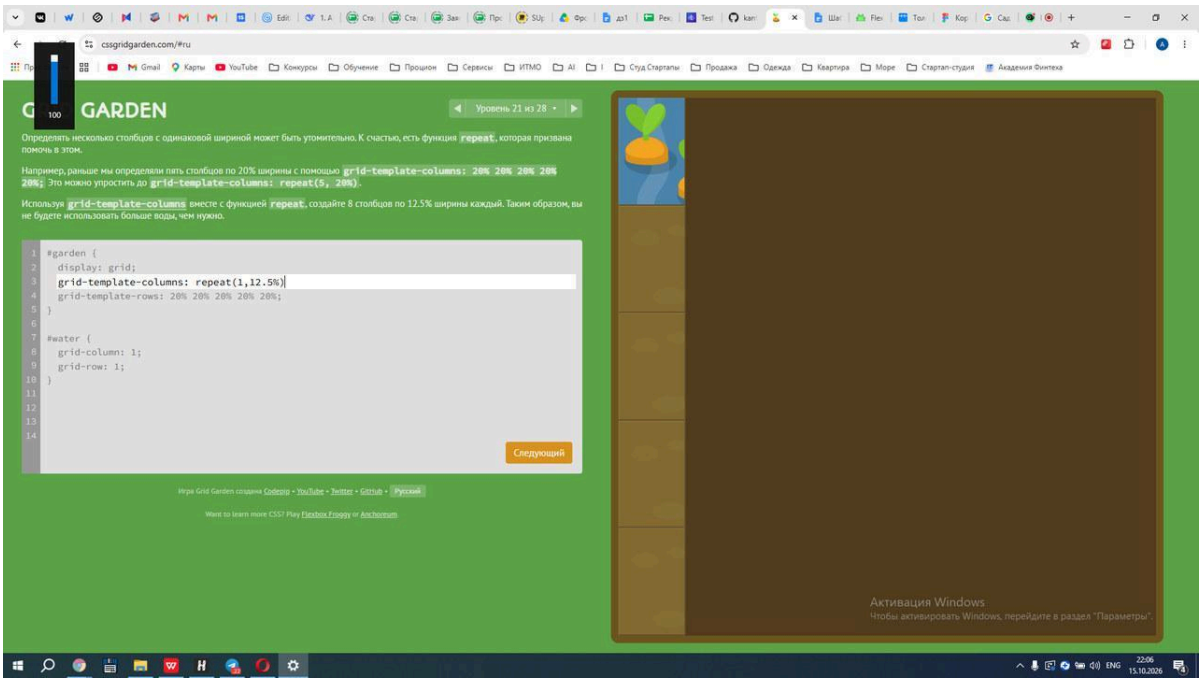


Рисунок 45 — CSS Grid Garden, уровень 21

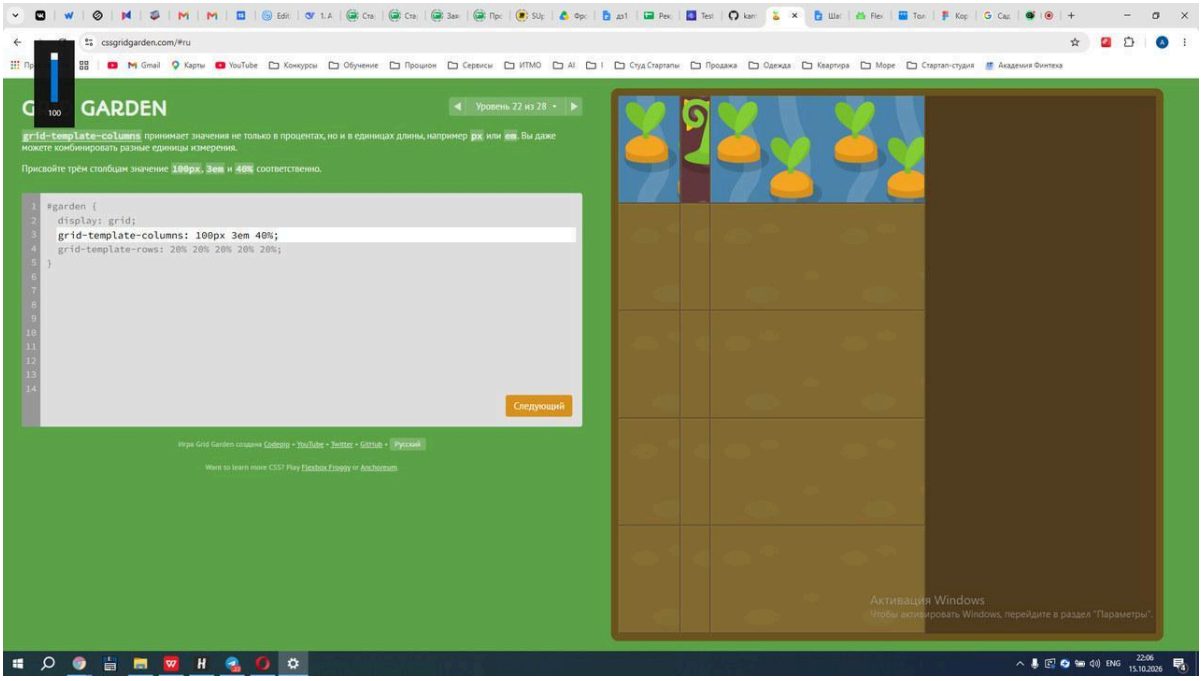


Рисунок 46 — CSS Grid Garden, уровень 22

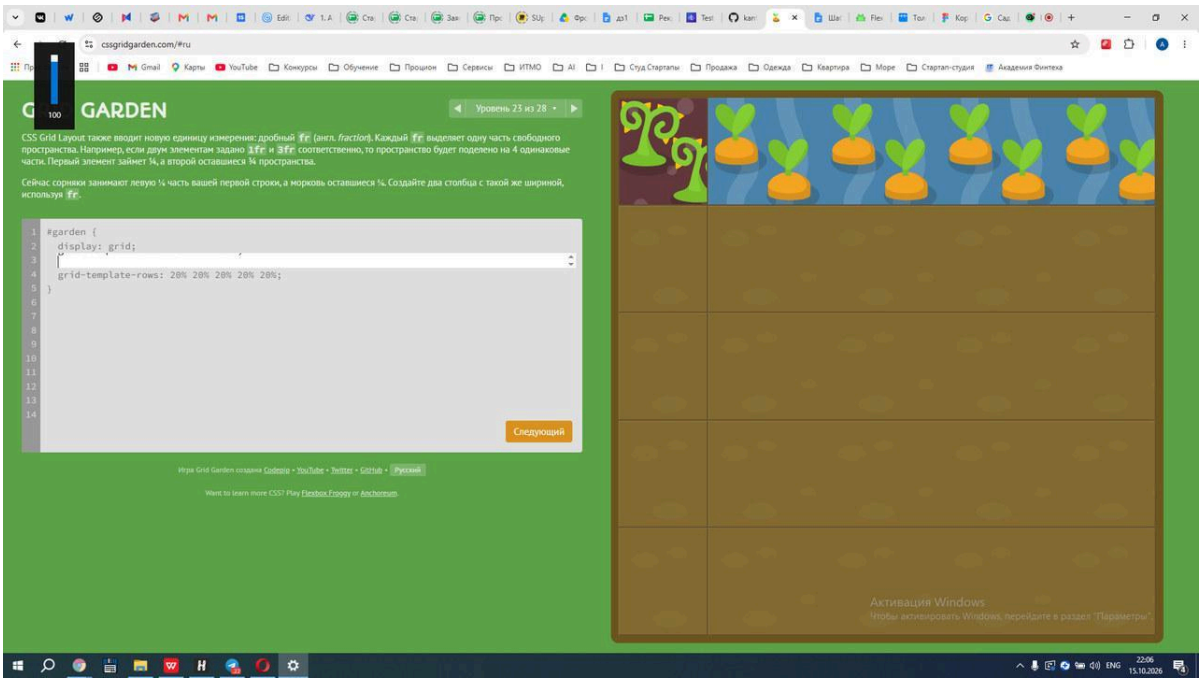


Рисунок 47 — CSS Grid Garden, уровень 23

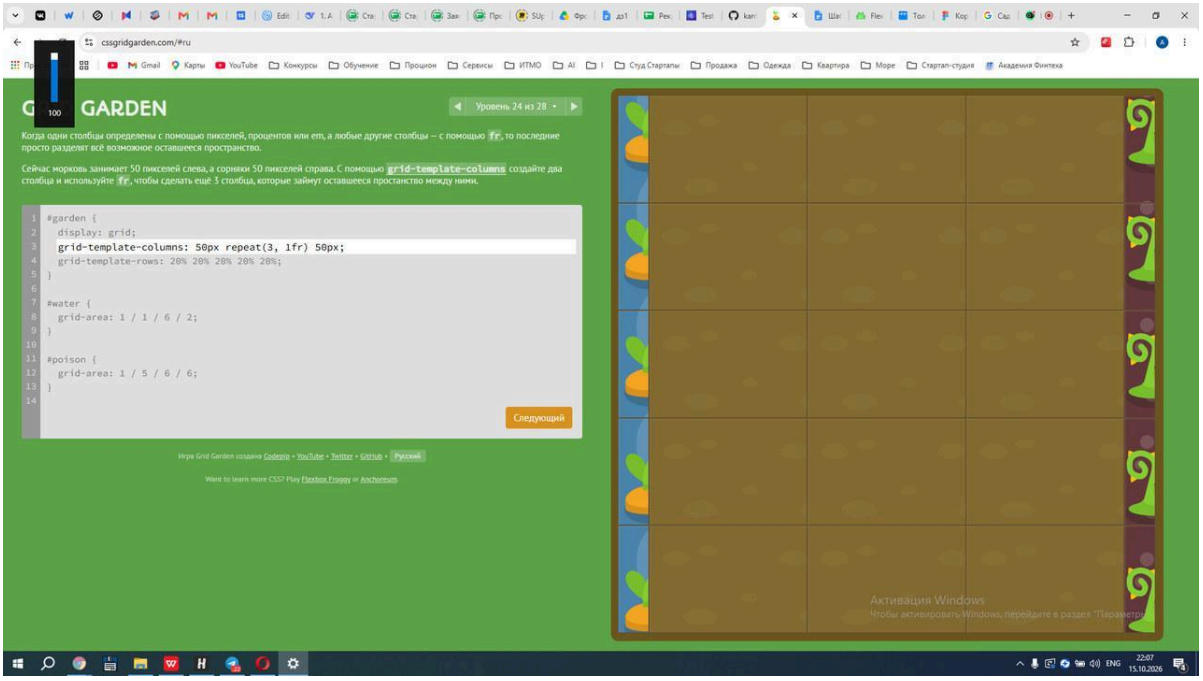


Рисунок 48 — CSS Grid Garden, уровень 24

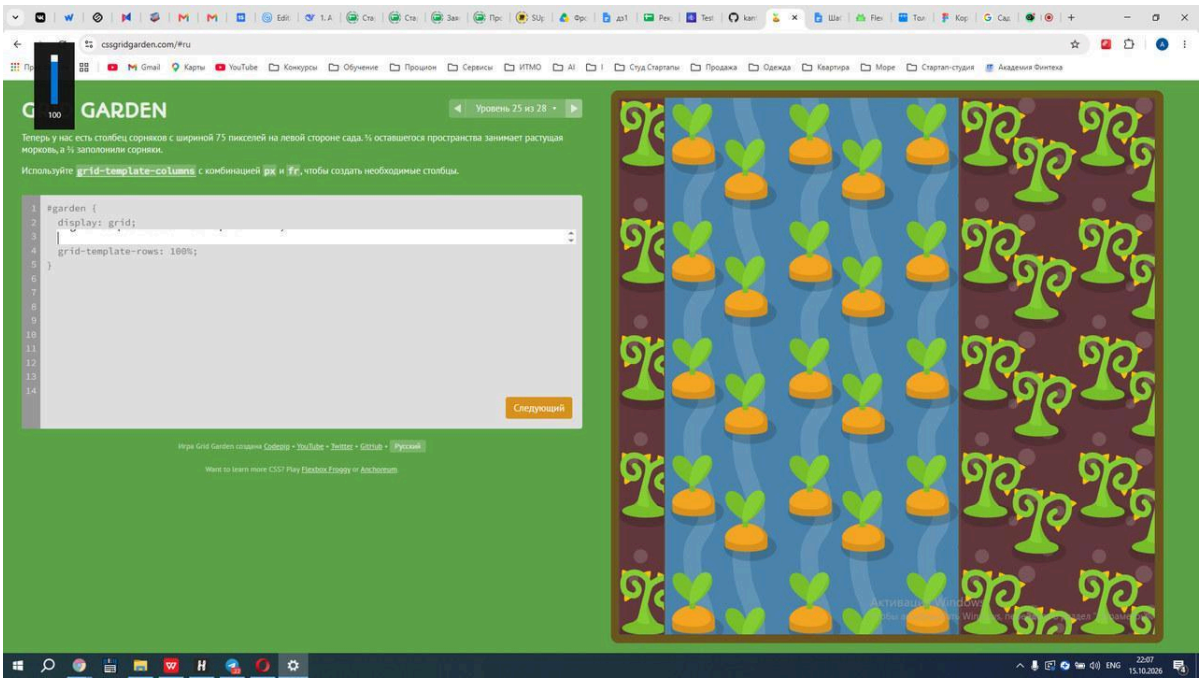


Рисунок 49 — CSS Grid Garden, уровень 25

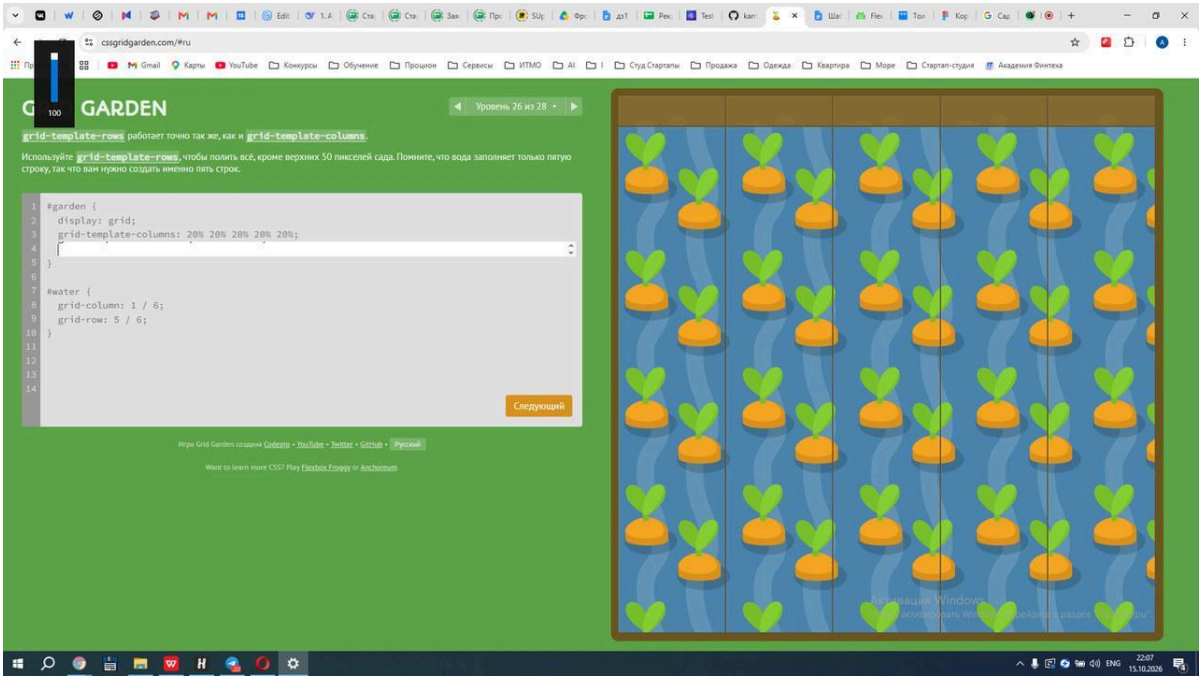


Рисунок 50 — CSS Grid Garden, уровень 26

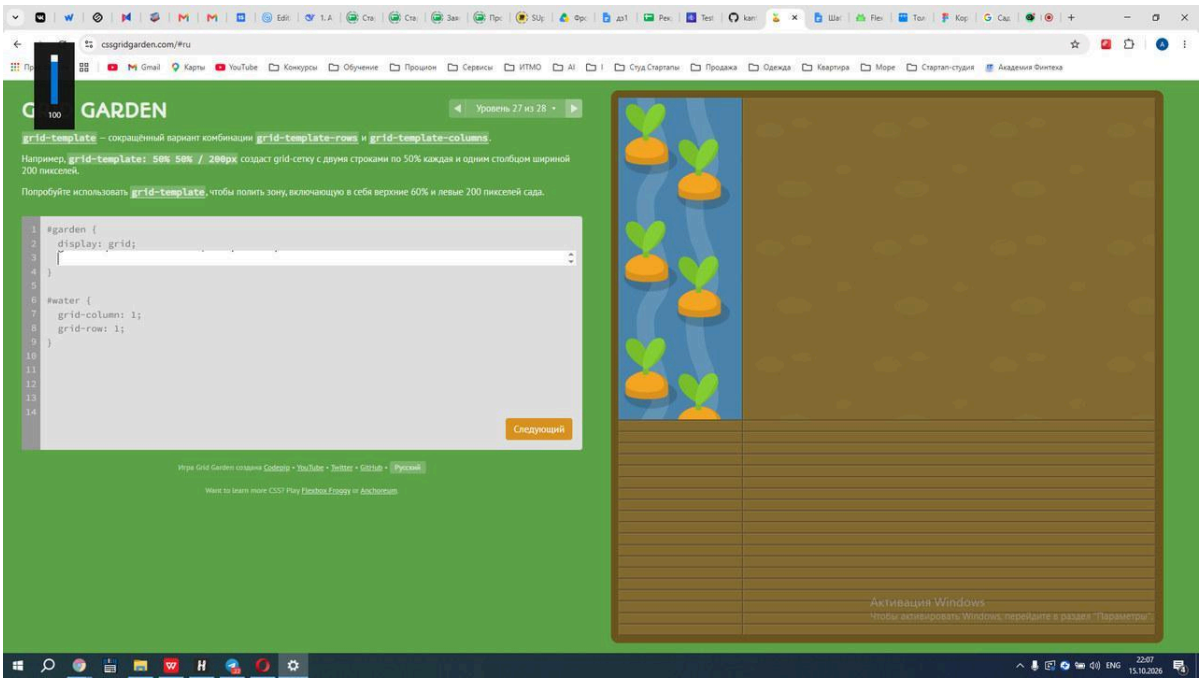


Рисунок 51 — CSS Grid Garden, уровень 27

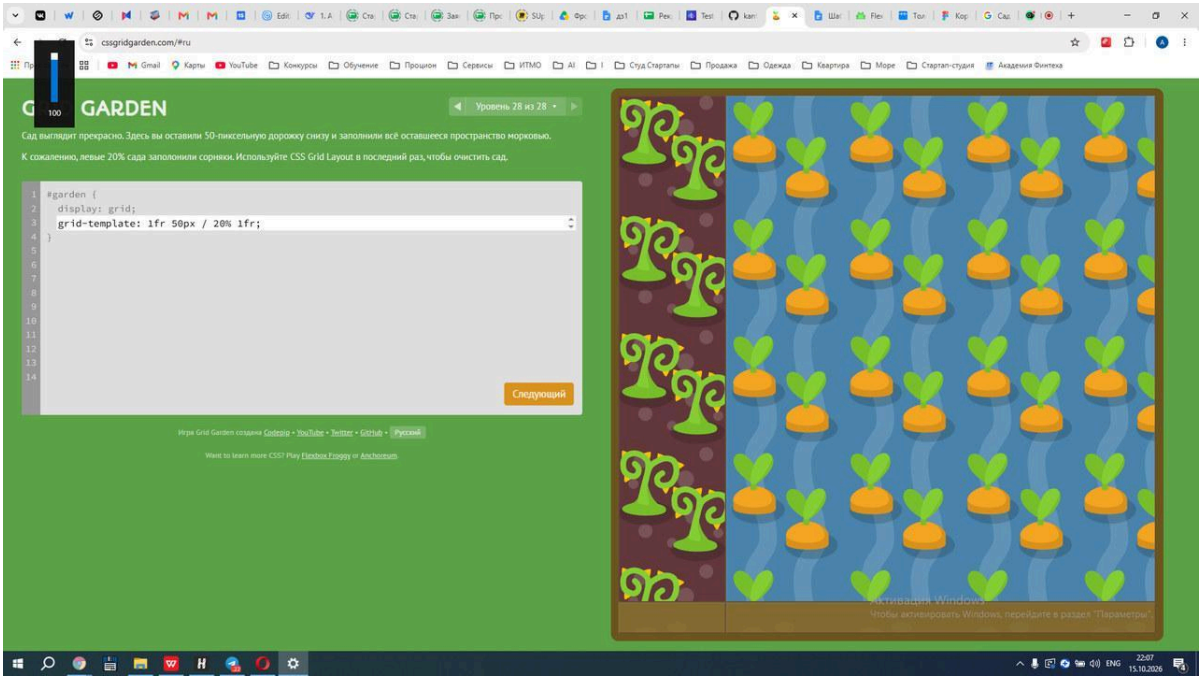


Рисунок 52 — CSS Grid Garden, уровень 28

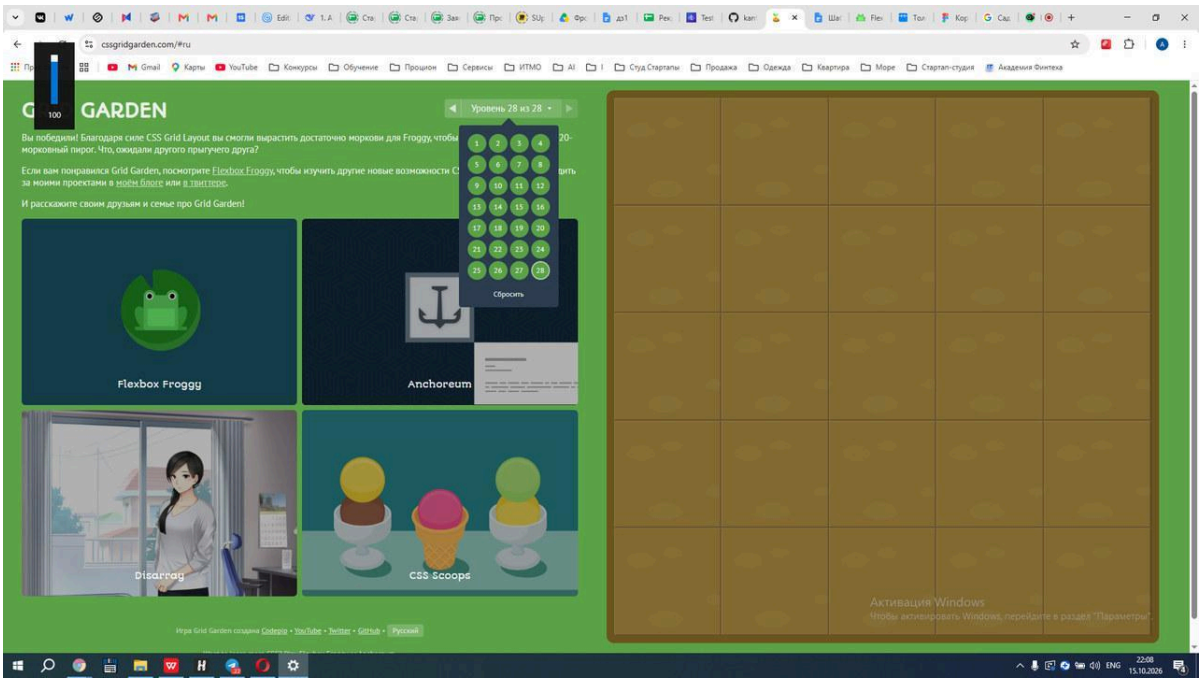


Рисунок 53 — CSS Grid Garden, итоговый экран после прохождения уровней 1–28

4. Основы Git: тренажёр Learn Git Branching

4.1. Описание тренажёра

Learn Git Branching — интерактивная обучающая среда, в которой история репозитория представлена в виде графа коммитов и веток. Пользователь вводит команды Git в консоль, после чего видит, как перемещаются указатели веток, создаются новые коммиты и изменяется состояние локального и удалённого репозиториев.

По предоставленным скриншотам зафиксированы первые четыре базовых уровня (знакомство с commit, ветвление, слияние и rebase), затем задания разделов по удалённым репозиториям и расширенные задания по push/pull. Один из исходных скриншотов показывает промежуточный уровень о detached HEAD; он сохранён в отчёте, но отдельно не засчитывается в перечисление обязательных уроков.

4.2. Изученные команды Git

Команда / понятие	**Назначение и пояснение**
git commit	Создаёт коммит из подготовленных изменений. Коммит фиксирует состояние проекта в истории.
git branch <имя>	Создаёт новую ветку, указывающую на текущий коммит; сама по себе команда не переключает рабочую ветку.
git checkout <ветка>	Переключает рабочее состояние на указанную ветку или коммит.
git checkout -b <имя>	Создаёт новую ветку и сразу переключается на неё.
git merge <ветка>	Объединяет изменения указанной ветки с текущей. В зависимости от истории Git может выполнить fast-forward или создать merge-коммит.
git rebase <ветка>	Переносит коммиты текущей ветки поверх указанной базовой ветки, формируя новую линейную историю и новые идентификаторы коммитов.
git clone <URL>	Создаёт локальную копию удалённого репозитория и настраивает связь с источником.
git fetch <remote>	Загружает новые объекты и обновляет ссылки на удалённые ветки, но не объединяет их автоматически с текущей локальной веткой.
git pull	Получает изменения из удалённого репозитория и интегрирует их в текущую ветку; тип интеграции зависит от настроек и аргументов.
git pull --rebase	Получает удалённые изменения, затем переносит локальные коммиты поверх обновлённой удалённой истории.
git push	Отправляет локальные коммиты в удалённый репозиторий, если это разрешено правилами обновления ветки.

Команда / понятие	Назначение и пояснение
git push origin main	Отправляет ветку main на удалённый репозиторий с именем origin.
git push origin <локальная>:<удалённая>	Явно указывает, какую локальную ссылку отправить в какую удалённую ветку, например c5:foo или foo:main.
git fetch origin <источник>:<назначение>	Загружает указанную ссылку и обновляет локальную ссылку назначения; формат применяется в заданиях на refspec.
git reset	Перемещает текущий указатель ветки на выбранный коммит. В зависимости от режима также может менять индекс и рабочие файлы.
git push origin :<ветка>	Пустой источник перед двоеточием в refspec используется для удаления указанной ветки в удалённом репозитории.
origin и o/main	origin — типичное имя удалённого репозитория; o/main на визуализации тренажёра обозначает ссылку на удалённую ветку main.

Дополнительные команды среды тренажёра

В консоли тренажёра также встречаются служебные команды delay, show goal и levels. Они используются для задержки анимации, отображения целевого состояния и просмотра списка уровней. Это команды интерфейса Learn Git Branching, а не стандартные команды Git. Аналогично git fakeTeamwork — специальная учебная команда для имитации действий другого участника, а не обычная команда Git.

4.3. Результат прохождения

Результаты показывают освоение базовой модели веток и коммитов, а также операций обмена изменениями между локальным и удалённым репозиториями. В базовых заданиях отрабатываются создание ветки, переключение, merge и rebase; в удалённых — clone, fetch, pull, push, обновление ссылок, явное указание источника и назначения, а также различия между локальной и удалённой веткой. На итоговом экране тренажёра видны отметки завершения уровней обоих разделов по удалённым репозиториям.

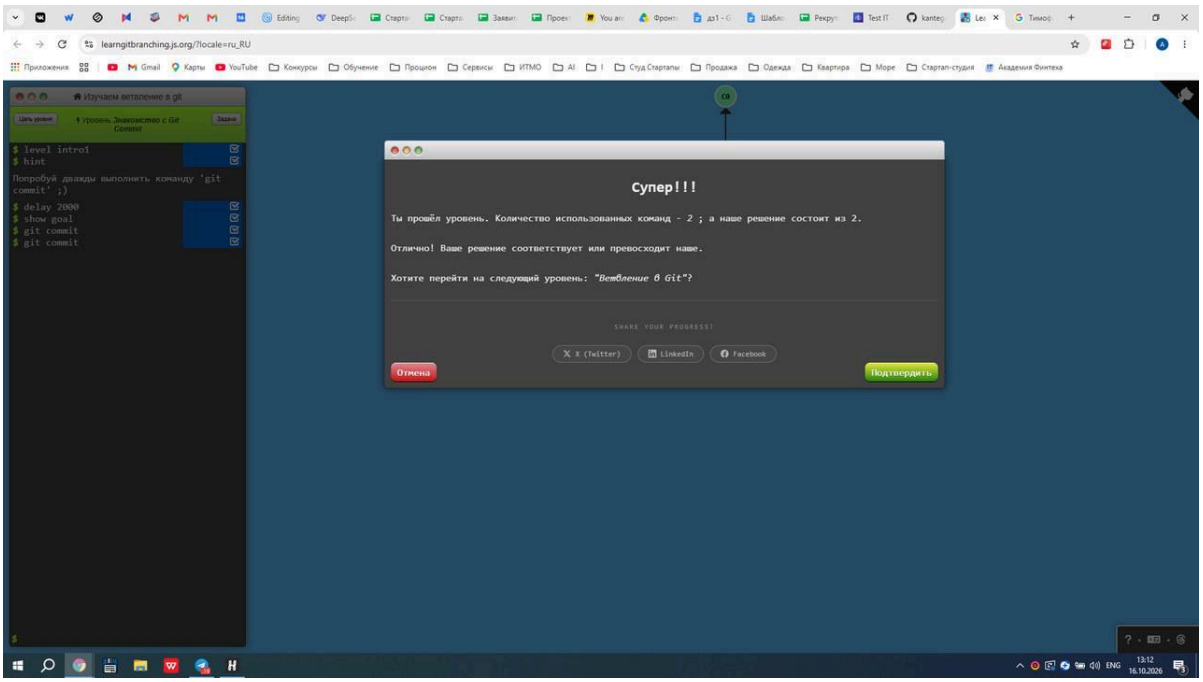


Рисунок 54 — Learn Git Branching, Intro 1 — «Знакомство с Git Commit»

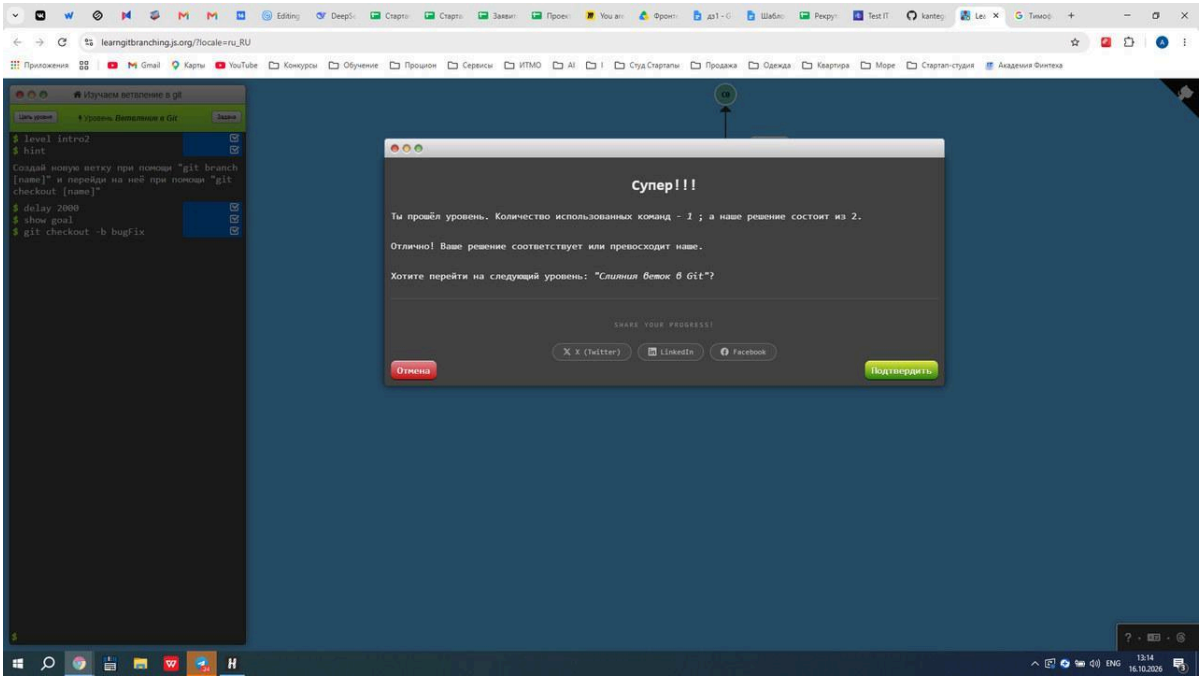


Рисунок 55 — Learn Git Branching, Intro 2 — «Ветвление в Git»

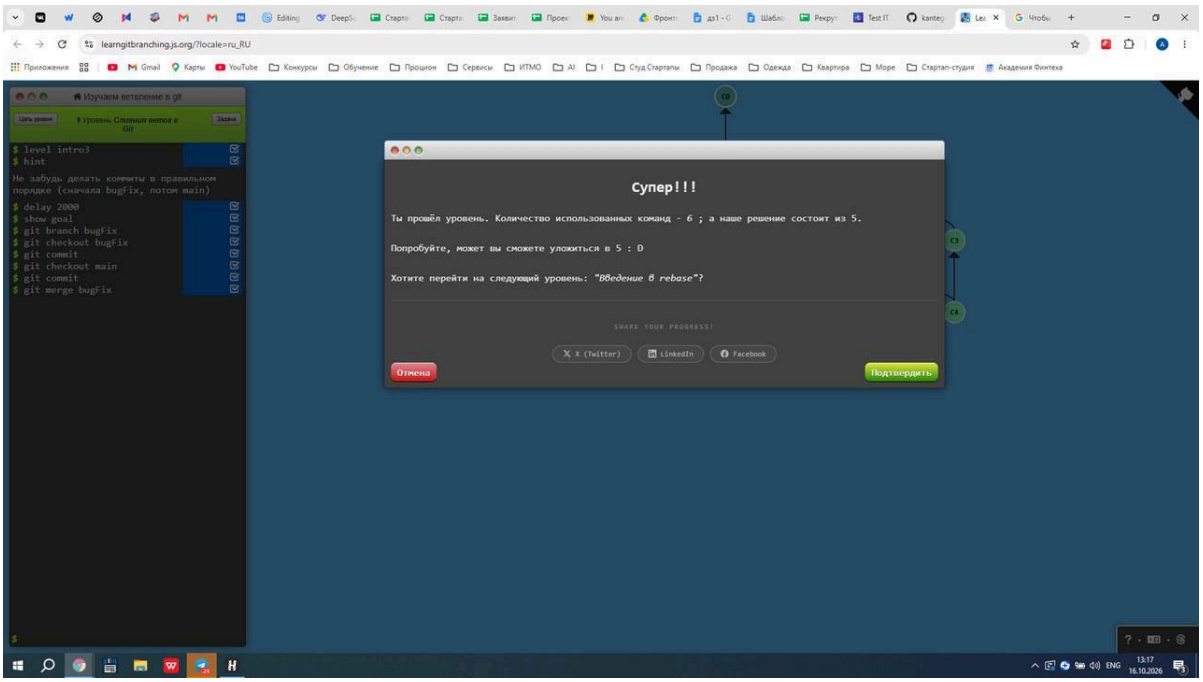


Рисунок 56 — Learn Git Branching, Intro 3 — «Слияния веток в Git»

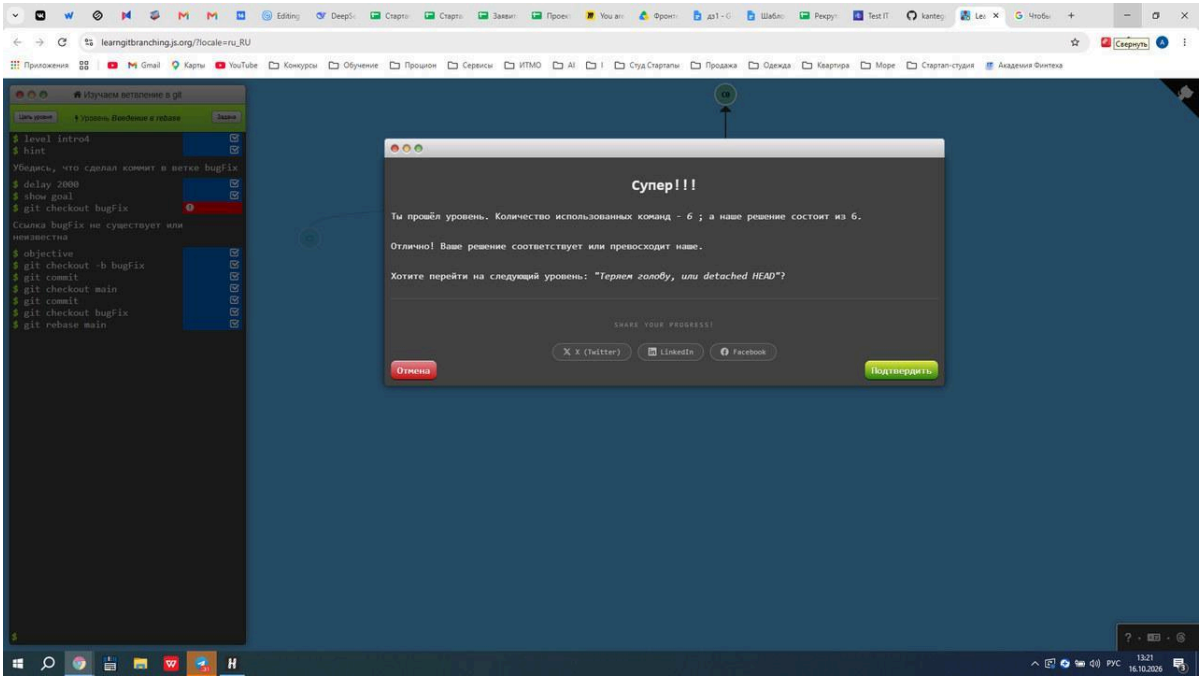


Рисунок 57 — Learn Git Branching, Intro 4 — «Введение в rebase»

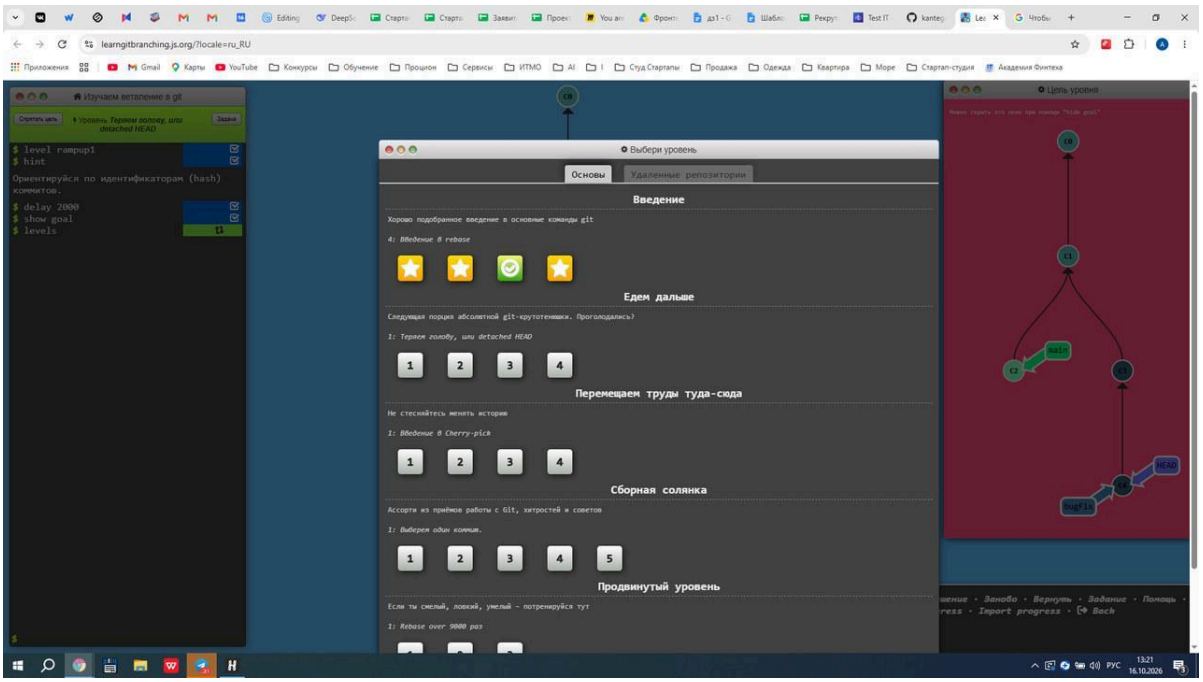


Рисунок 58 — Learn Git Branching, Rampup 1 — «Терминология, или detached HEAD» (промежуточный экран)

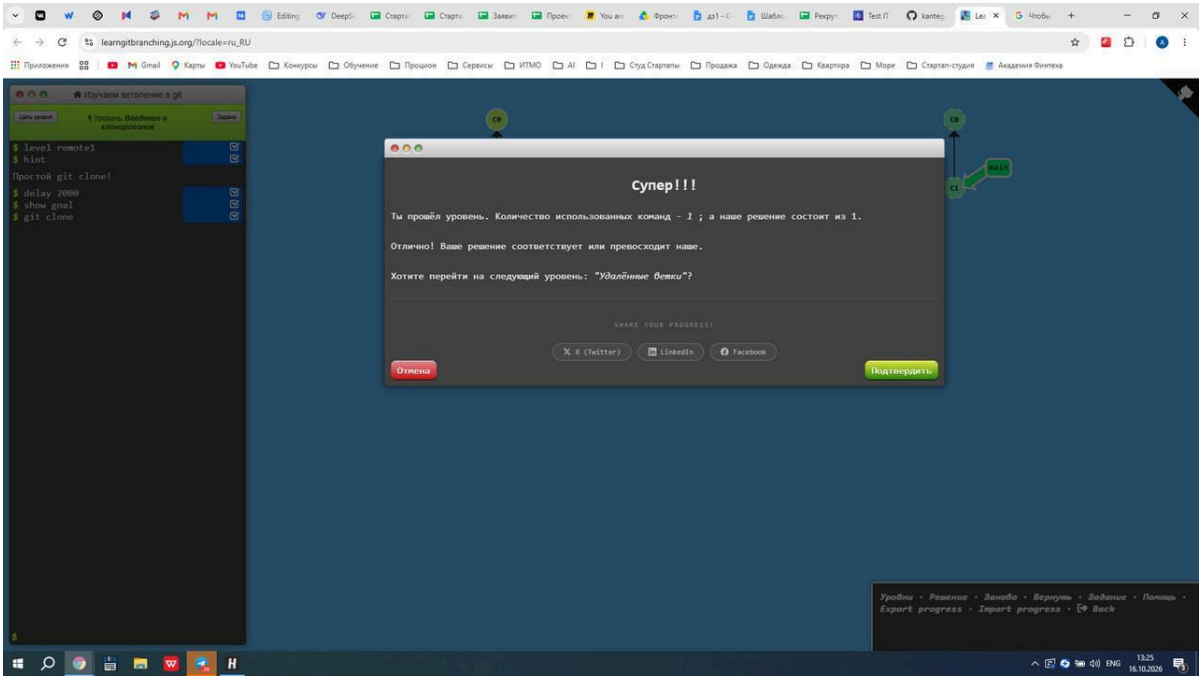


Рисунок 59 — Learn Git Branching, Remote 1 — «Введение в удалённые репозитории»

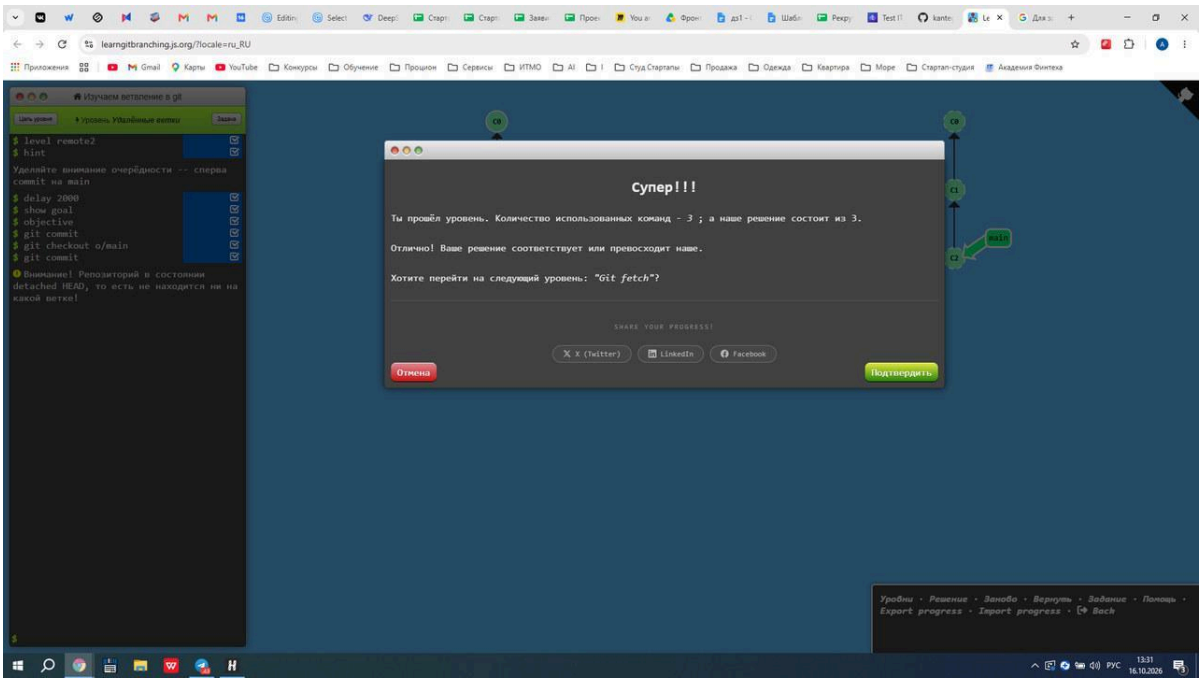


Рисунок 60 — Learn Git Branching, Remote 2 — «Удалённые ссылки»

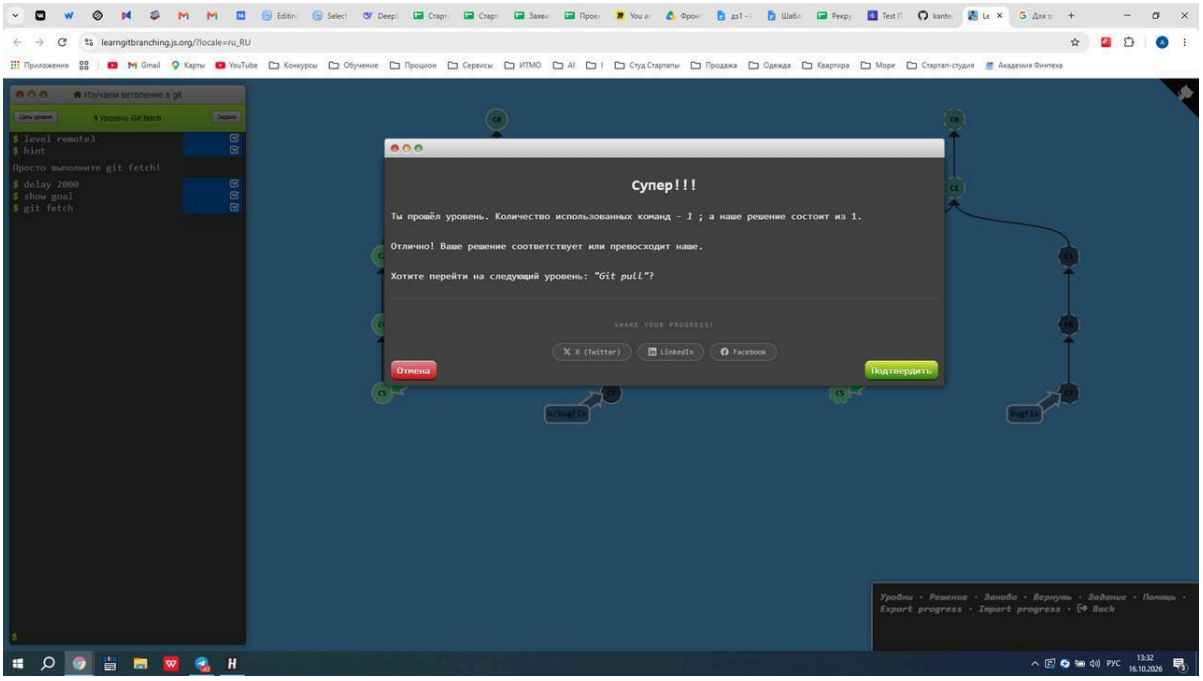


Рисунок 61 — Learn Git Branching, Remote 3 — «Git fetch»

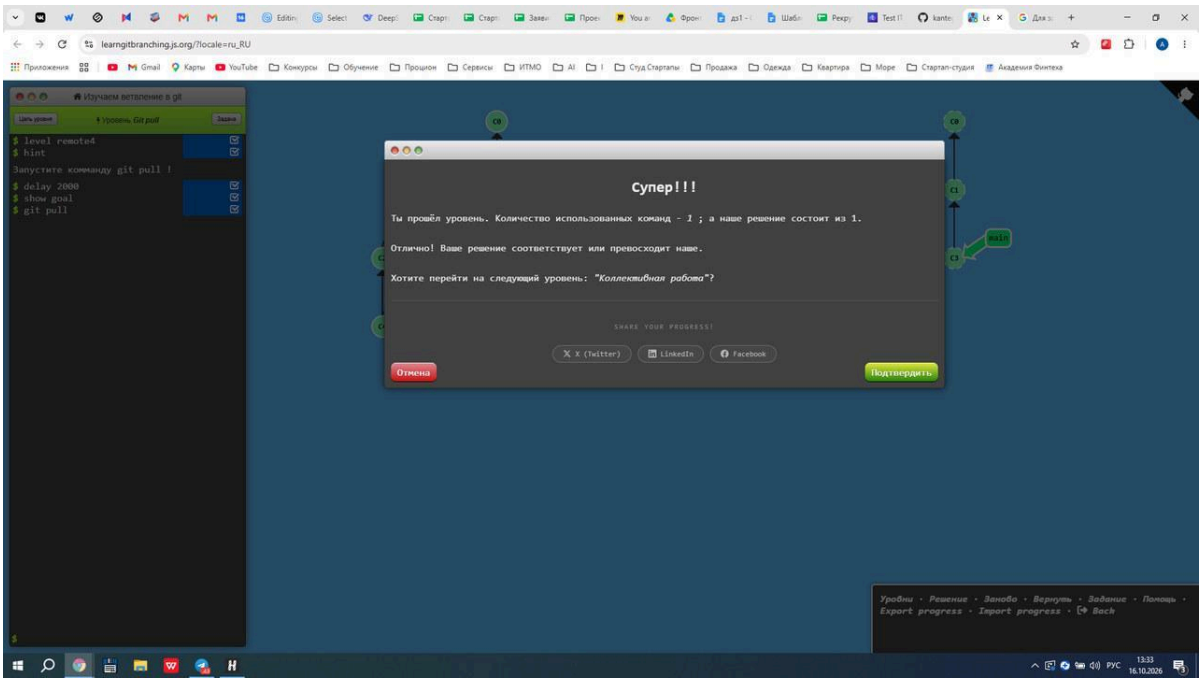


Рисунок 62 — Learn Git Branching, Remote 4 — «Git pull»

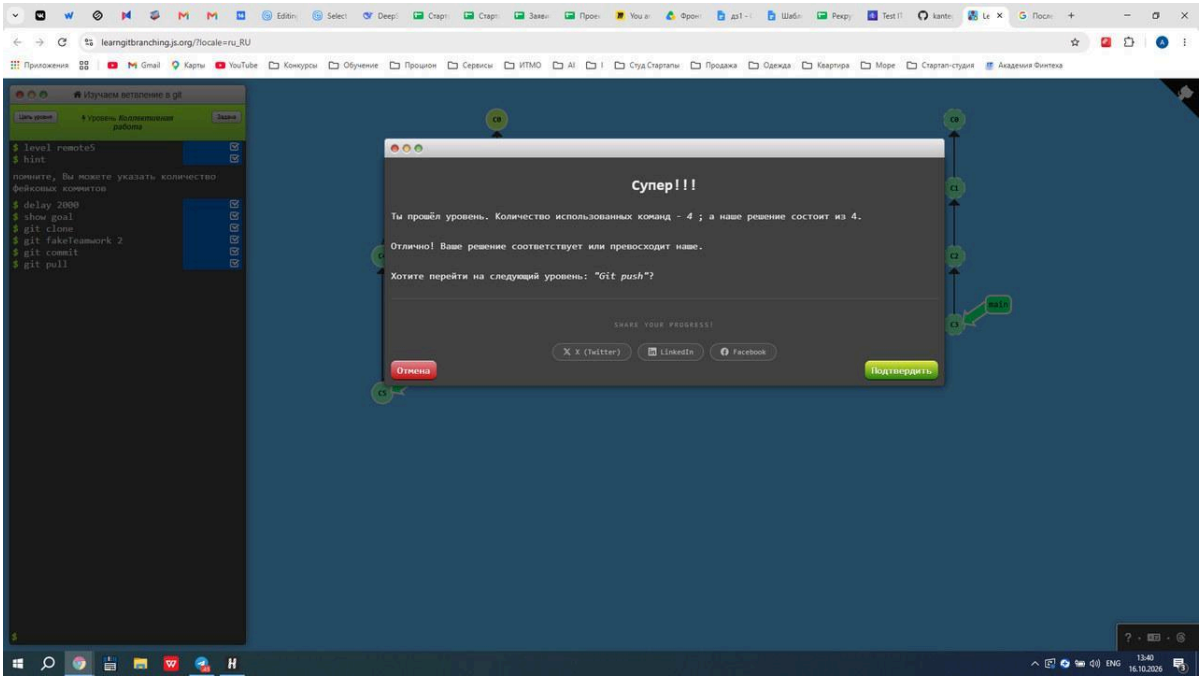


Рисунок 63 — Learn Git Branching, Remote 5 — «Коллективная работа»

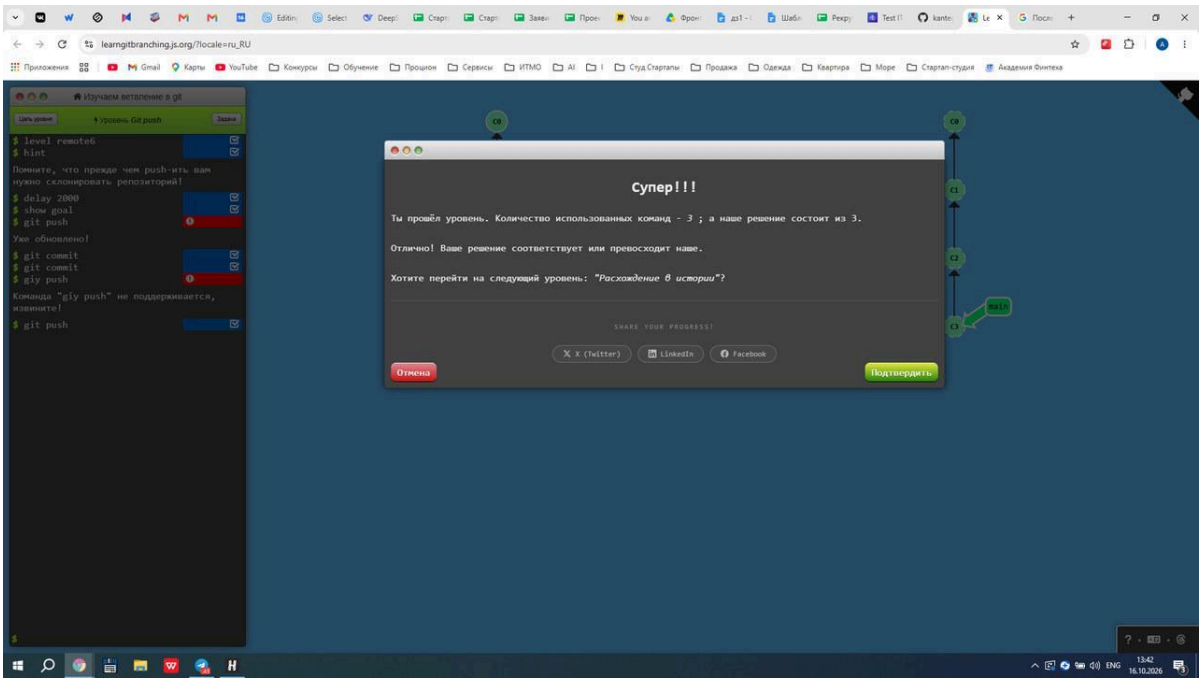


Рисунок 64 — Learn Git Branching, Remote 6 — «Git push»

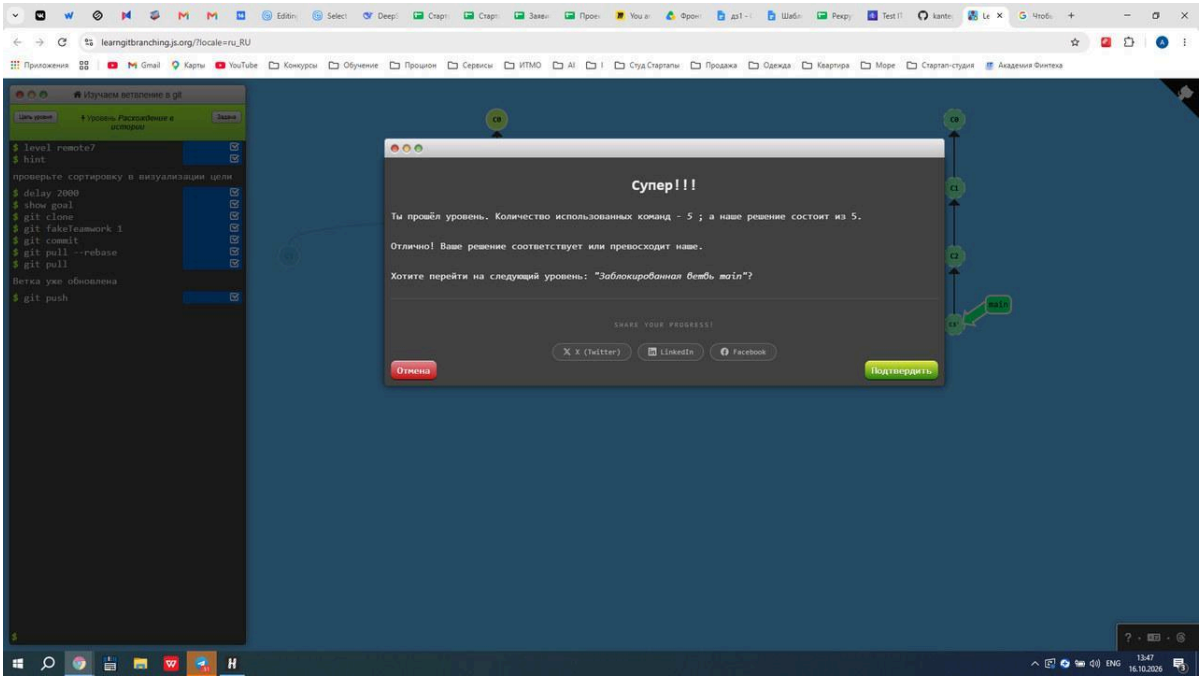


Рисунок 65 — Learn Git Branching, Remote 7 — «Расхождение в истории»

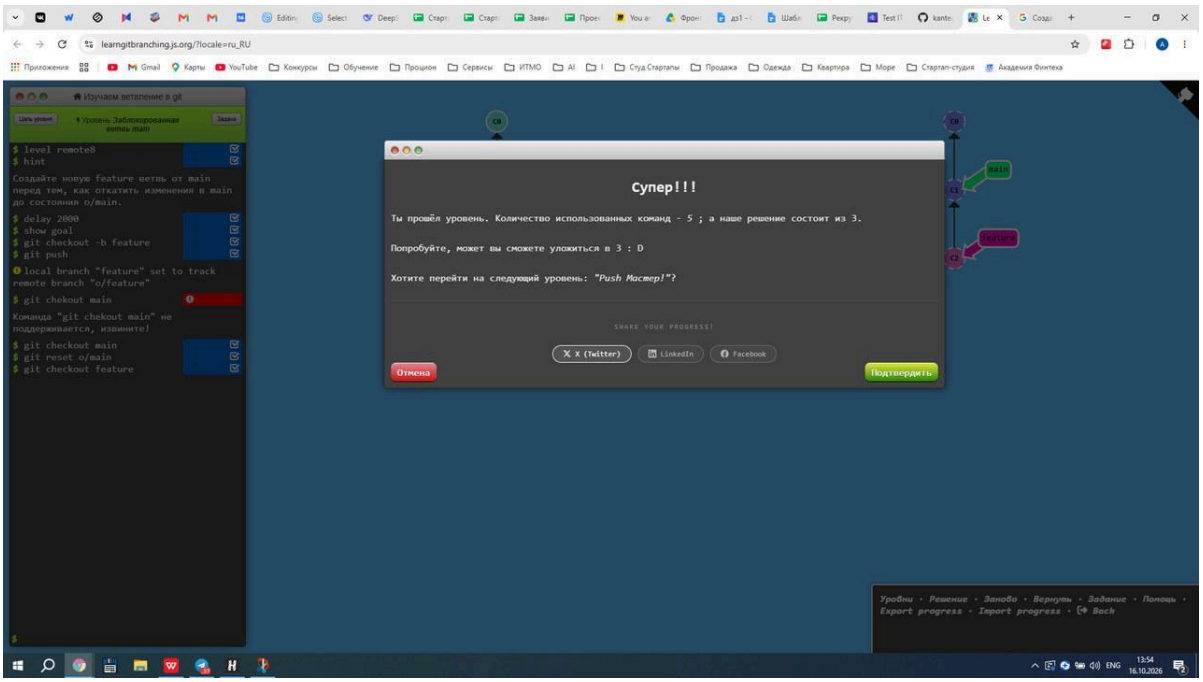


Рисунок 66 — Learn Git Branching, Remote 8 — «Заблокированная ветка main»

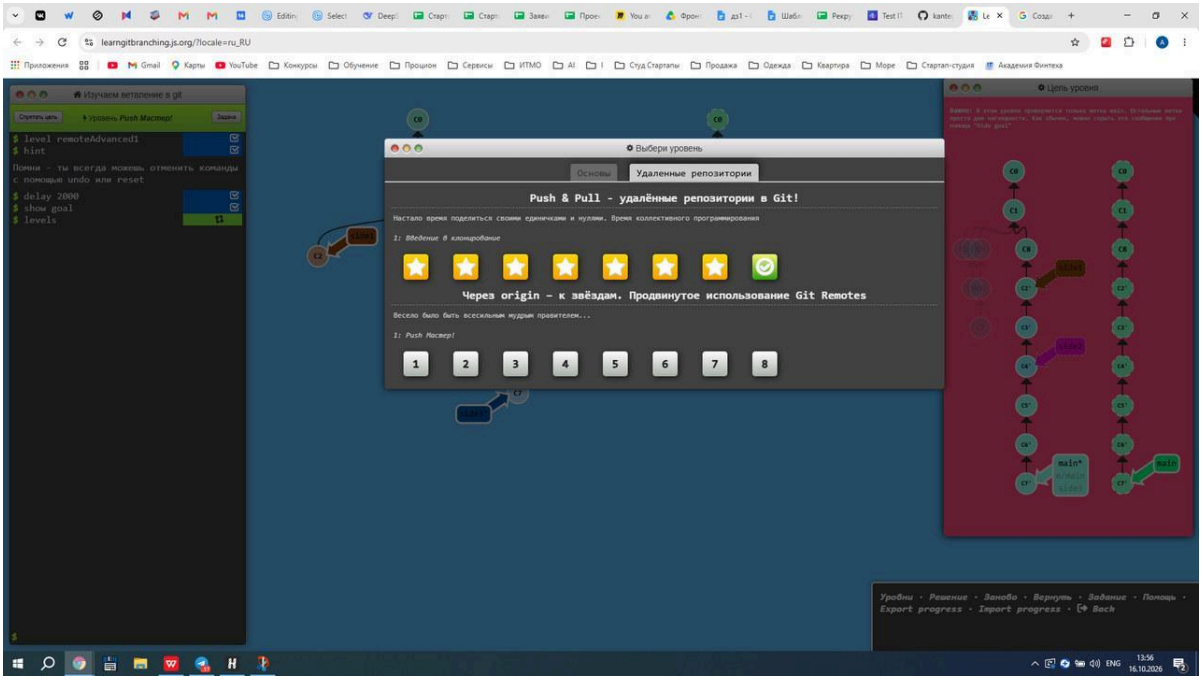


Рисунок 67 — Learn Git Branching, экран выбора уровней раздела Push & Pull / переход к расширенным заданиям

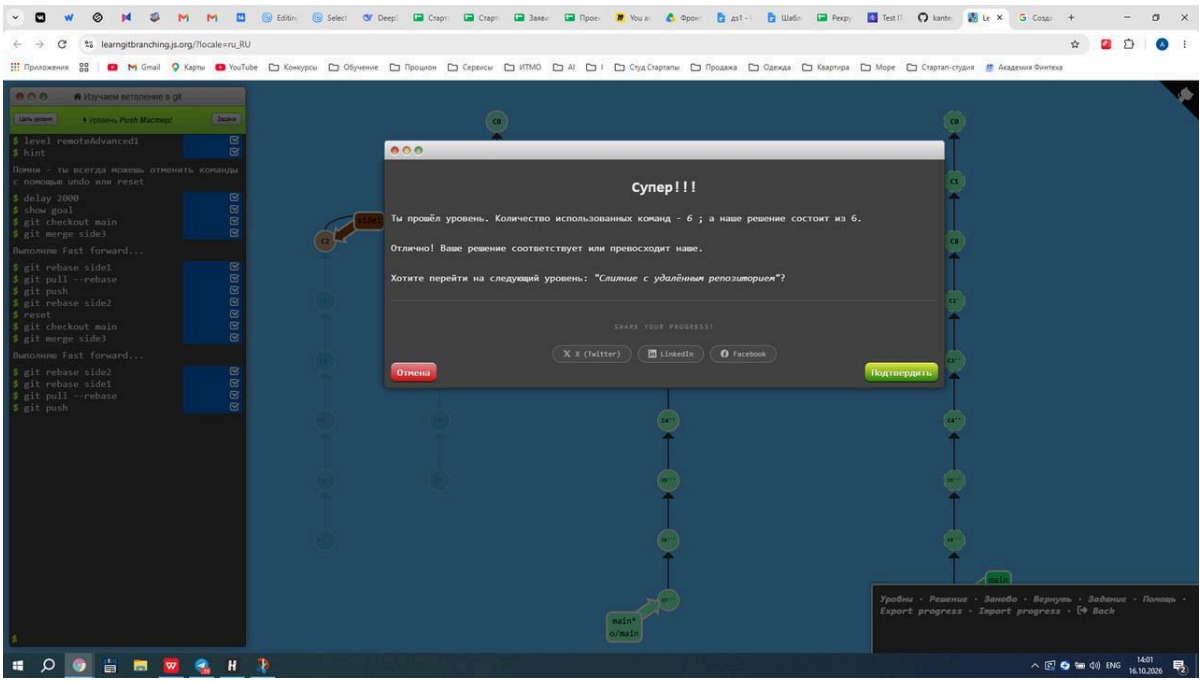


Рисунок 68 — Learn Git Branching, Remote Advanced 1 — «Push Master!»

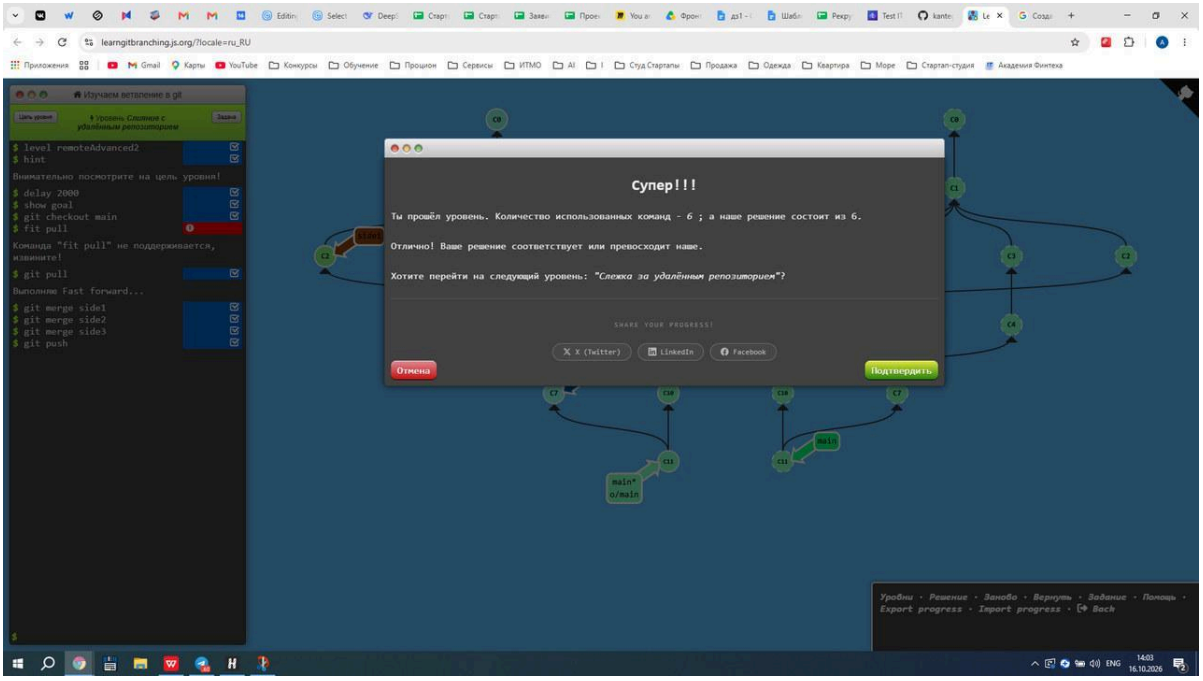


Рисунок 69 — Learn Git Branching, Remote Advanced 2 — «Слияние с удалённым репозиторием»

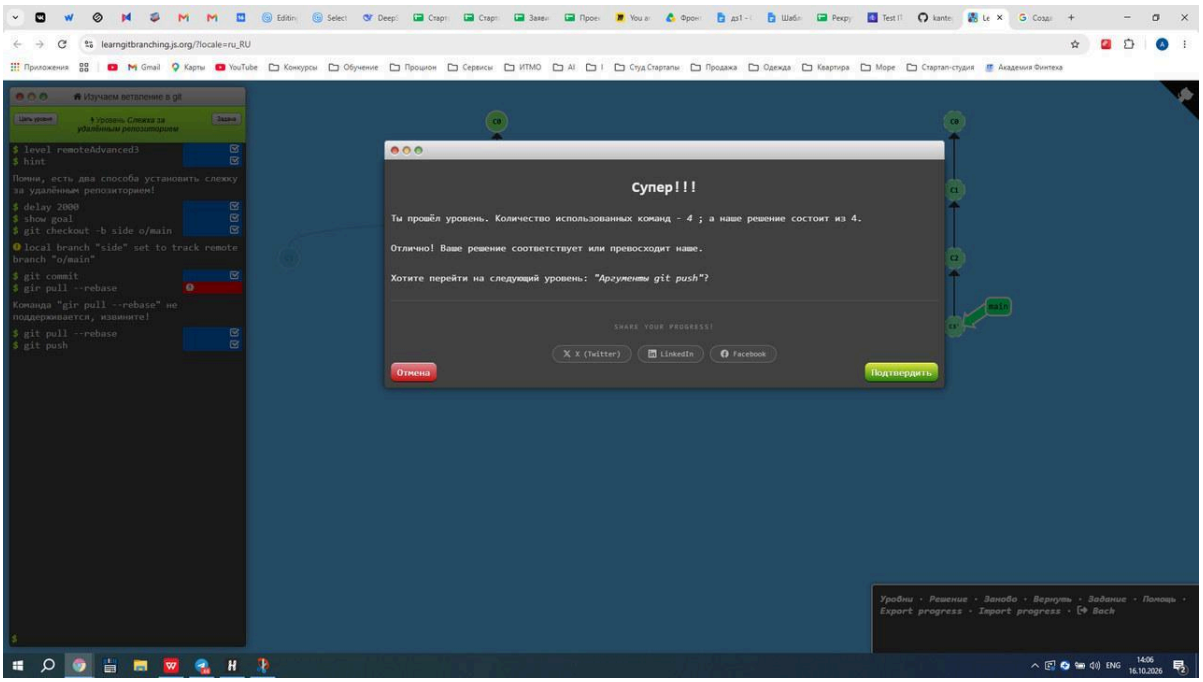


Рисунок 70 — Learn Git Branching, Remote Advanced 3 — «Слежка за удалённым репозиторием»

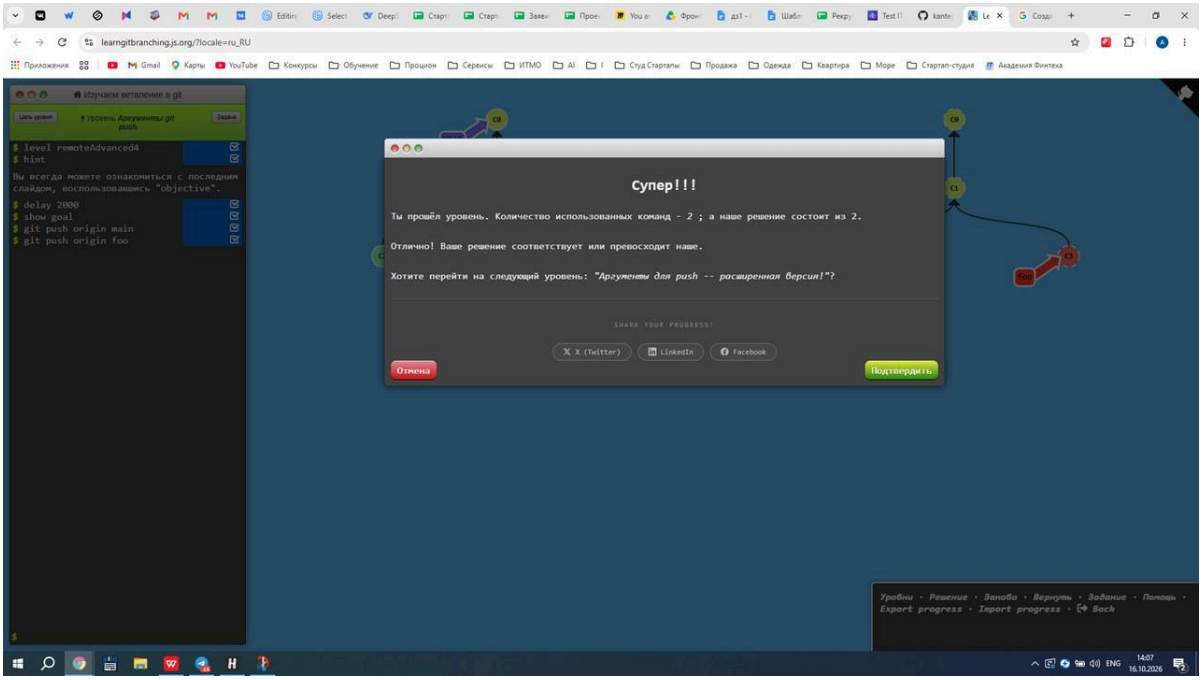


Рисунок 71 — Learn Git Branching, Remote Advanced 4 — «Аргументы git push»

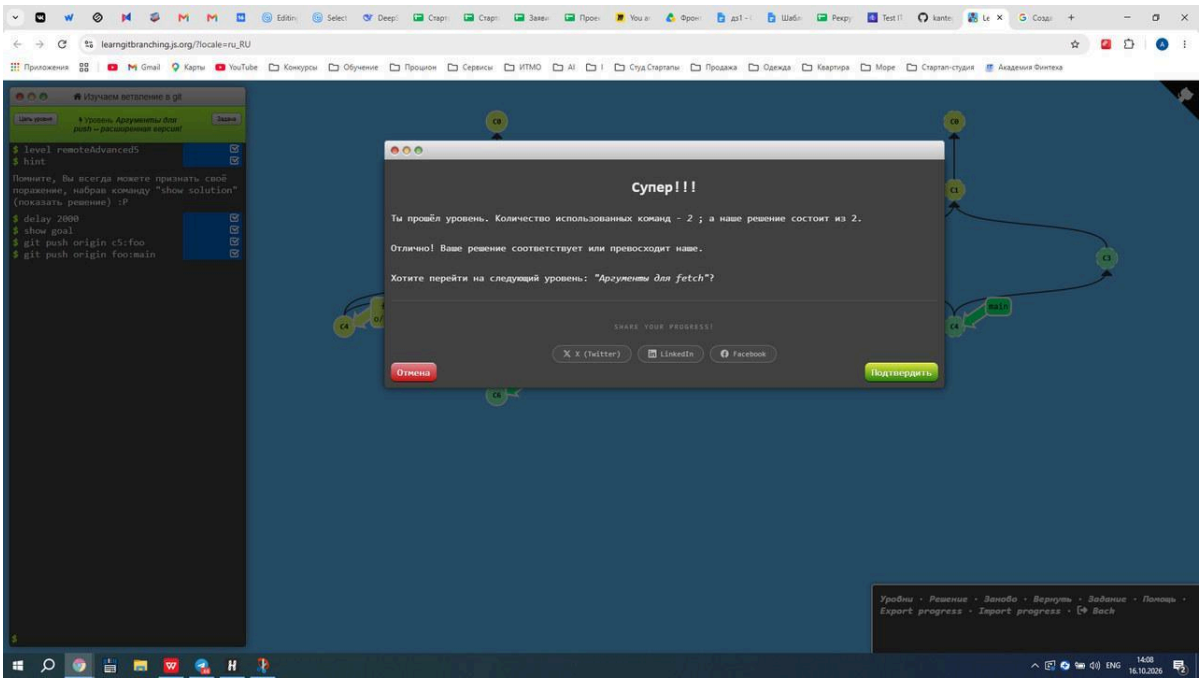


Рисунок 72 — Learn Git Branching, Remote Advanced 5 — «Аргументы для push — расширенная версия»

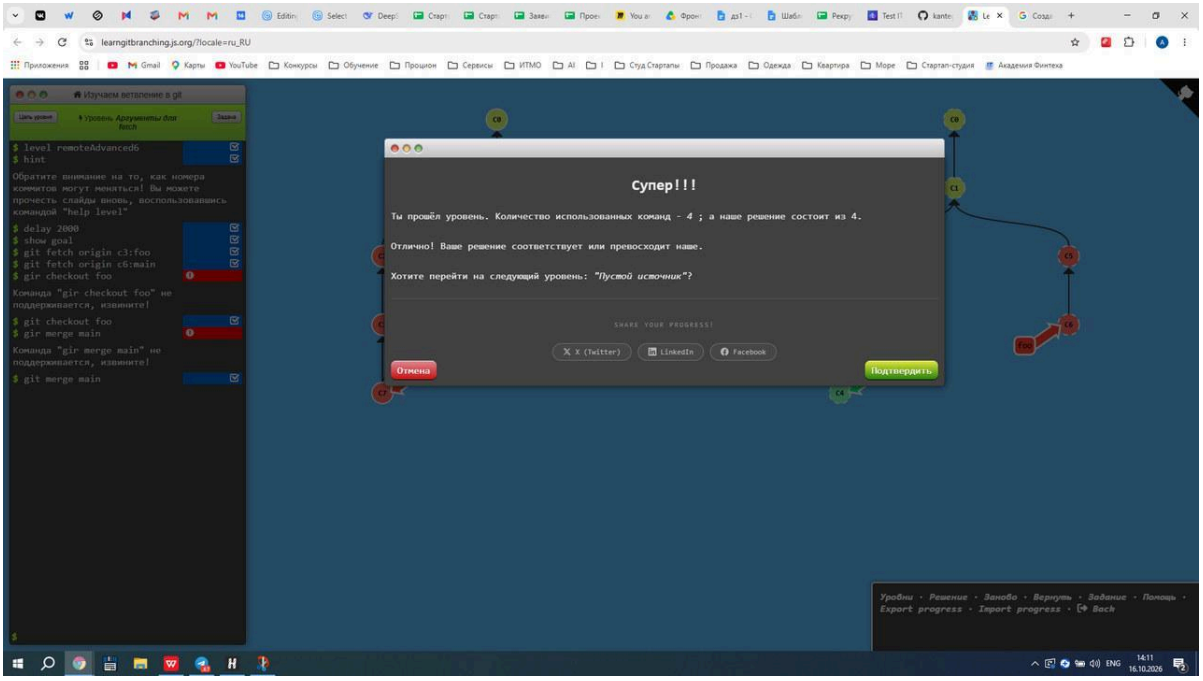


Рисунок 73 — Learn Git Branching, Remote Advanced 6 — «Аргументы для fetch»

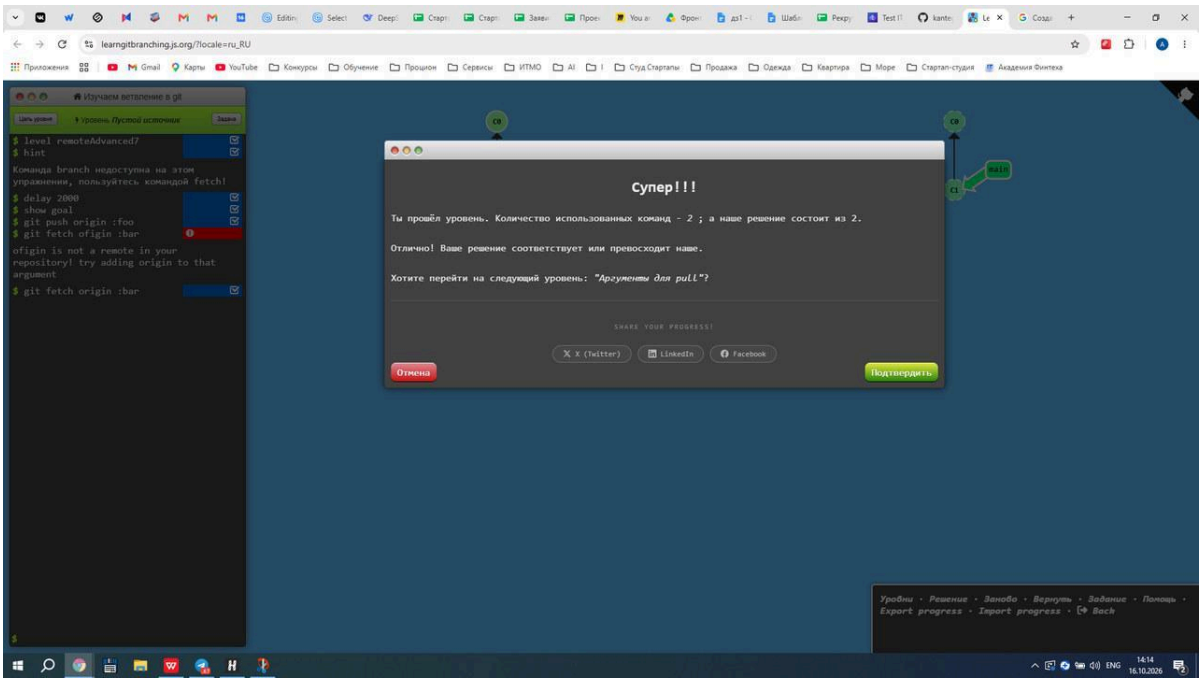


Рисунок 74 — Learn Git Branching, Remote Advanced 7 — «Пустой источник»

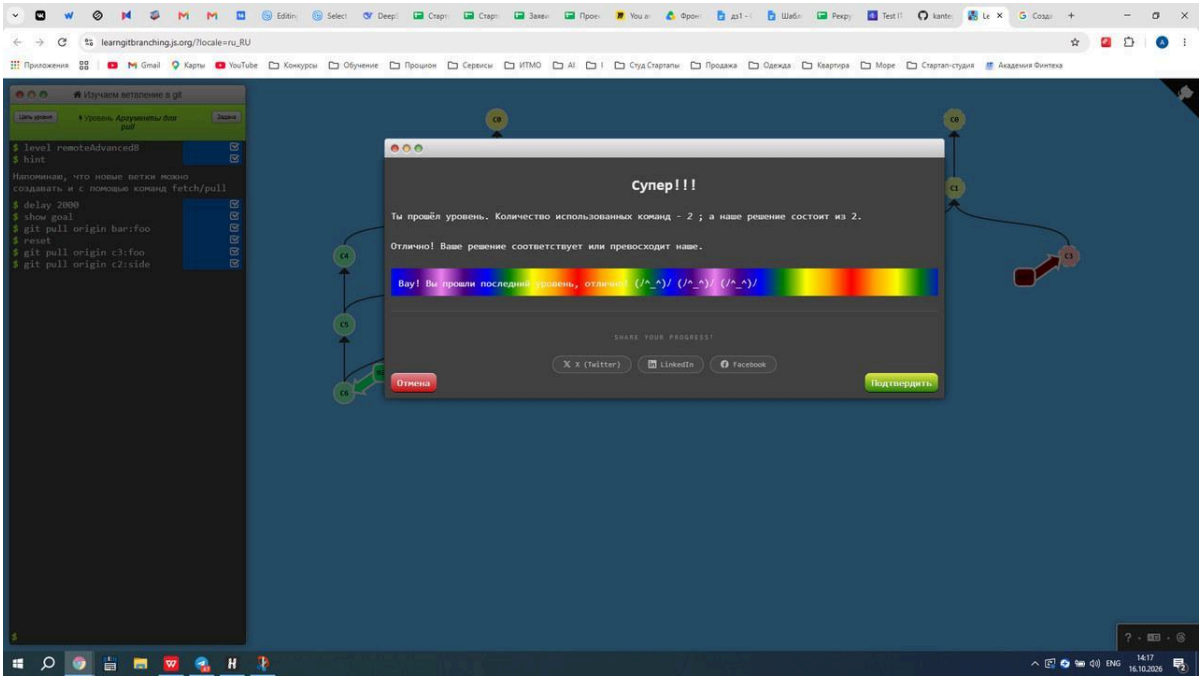


Рисунок 75 — Learn Git Branching, Remote Advanced 8 — «Аргументы для pull»

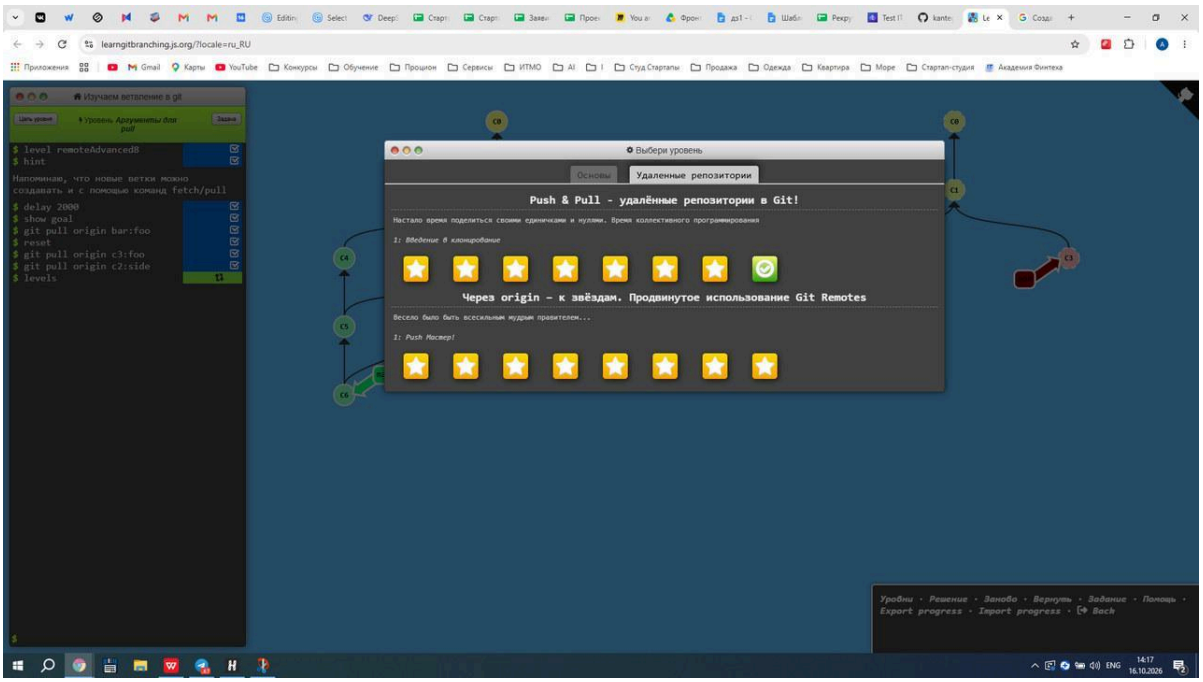


Рисунок 76 — Learn Git Branching, итоговый экран завершения разделов по удалённым репозиториям

5. Заключение

В результате выполнения домашней работы закреплены принципы построения раскладок средствами CSS Flexbox и CSS Grid. Flexbox позволяет гибко распределять элементы по главной и поперечной осям, менять направление, порядок и перенос элементов. Grid позволяет точно задавать положение элементов по линиям сетки, объединять строки и столбцы в области и управлять размером треков.

Работа с Learn Git Branching позволила отработать операции над историей коммитов и ветками, а также базовый цикл взаимодействия с удалённым репозиторием: клонирование, получение и отправка изменений, интеграцию изменений через merge или rebase. Все предоставленные скриншоты включены в отчёт с нумерацией и подписями, соответствующими уровням и итоговым экранам.